

JAPAN TRAVEL AI BLOGGER

Gabriele Florio – Enricomaria Di Rosolini

Sommario

JAPAN TRAVEL AI BLOGGER	1
1. Introduzione	3
2. Architettura	4
2.1 Gestione dello stato.....	6
2.2 Nodo Planner	8
2.3 Nodo Research.....	10
2.4 Nodo Recap	11
2.5 Nodo Fact checker	13
2.6 Nodo Drafter	14
2.7 Nodo human feedback.....	15
2.8 Nodo kg updater	16
3. Tool	17
3.1 Advanced web research	17
3.2 Kg query tool	18
3.3 Rag retrieval tool.....	19
3.4 Travel route tool.....	20
3.5 Fact checking tool	21
4. Knowledge Graph	23
5. RAG	25
6. Fine Tuning.....	27
Baseline	28
Metriche di Valutazione Utilizzate.....	28
Struttura e Generazione del Dataset	29
Suddivisione e Bilanciamento del Dataset	29
6.5 La strategia di fine-tuning	30
6.7 Confronto con la baseline e analisi dei risultati	32
7. Valutazione e Osservabilità del Sistema	36

Analisi qualitativa e casi d'uso	36
Caso d'uso 1 [Es. Guida]	36
Caso d'uso 2 [Es. Itinerario failure case]	43
Valutazione delle fonti e grounding	48
Analisi dei fallimenti (failure cases)	49
LangSmith	50

1. Introduzione

Questo progetto descrive la progettazione, lo sviluppo e la valutazione di un sistema di Intelligenza Artificiale basato su agenti (Agentic AI), realizzato nell'ambito del corso di CCAI 2025-2026 della Prof.ssa Daniela Giordano. Il sistema è stato progettato come un "Copilot" a supporto della gestione di un blog dedicato ai viaggi in Giappone. In particolare, l'obiettivo è assistere il blogger nelle diverse fasi del lavoro, dalla pianificazione del calendario editoriale alla ricerca delle informazioni, fino alla generazione delle bozze degli articoli e alla verifica dell'accuratezza dei contenuti. In questo modo, il sistema mira a facilitare la produzione di articoli affidabili, ben documentati e con contenuti sempre vari e di qualità.

Per orchestrare la logica decisionale e soddisfare i requisiti di gestione dello stato, l'architettura è stata implementata utilizzando il framework LangGraph. L'architettura è implementata come un grafo di stato orchestrato tramite il framework LangGraph, in cui ogni nodo rappresenta una fase distinta della pipeline. Il flusso si articola in una sequenza di agenti dedicati: un planner che definisce il calendario editoriale, un researcher che raccoglie i dati attraverso strumenti esterni, un recap che organizza le informazioni in un dossier tematico, un modulo di fact-checking, un drafter incaricato della scrittura dell'articolo e, infine, una fase di human-in-the-loop che consente la supervisione e l'approvazione umana prima della pubblicazione. Uno stato condiviso mantiene la coerenza delle informazioni scambiate tra i vari nodi lungo tutto il processo. Il sistema mantiene traccia del contesto ad ogni step, registrando le tracce di ragionamento (reasoning trace), l'input dell'utente e gli output degli strumenti utilizzati.

Un elemento cardine della soluzione proposta è l'integrazione di un approccio K-RAG. Il sistema unisce una memoria semantica strutturata, implementata tramite Neo4j, a un database vettoriale locale gestito tramite ChromaDB. Il Knowledge Graph (KG) non funge da semplice archivio passivo, ma viene attivamente interrogato dal modulo di pianificazione per analizzare lo storico degli articoli, identificare lacune di copertura geografica (coverage gaps) e formulare query di ricerca contestualizzate. Inoltre, il modulo RAG sfrutta un approccio ibrido (BM25 ed embeddings) e abilità multimodali, impiegando il modello CLIP per vettorializzare e recuperare automaticamente le immagini pertinenti da impaginare nell'articolo finale.

Il comportamento dell'agente segue un paradigma di ragionamento strutturato (stile ReAct), in cui la selezione degli strumenti viene effettuata dinamicamente e giustificata all'interno del processo decisionale. Oltre agli strumenti fondamentali per la ricerca web e l'interrogazione del K-RAG, l'agente è dotato di tool personalizzati, tra cui un modulo geospaziale per il calcolo logistico degli itinerari turistici (distanze Haversine e stima dei tempi di percorrenza in treno).

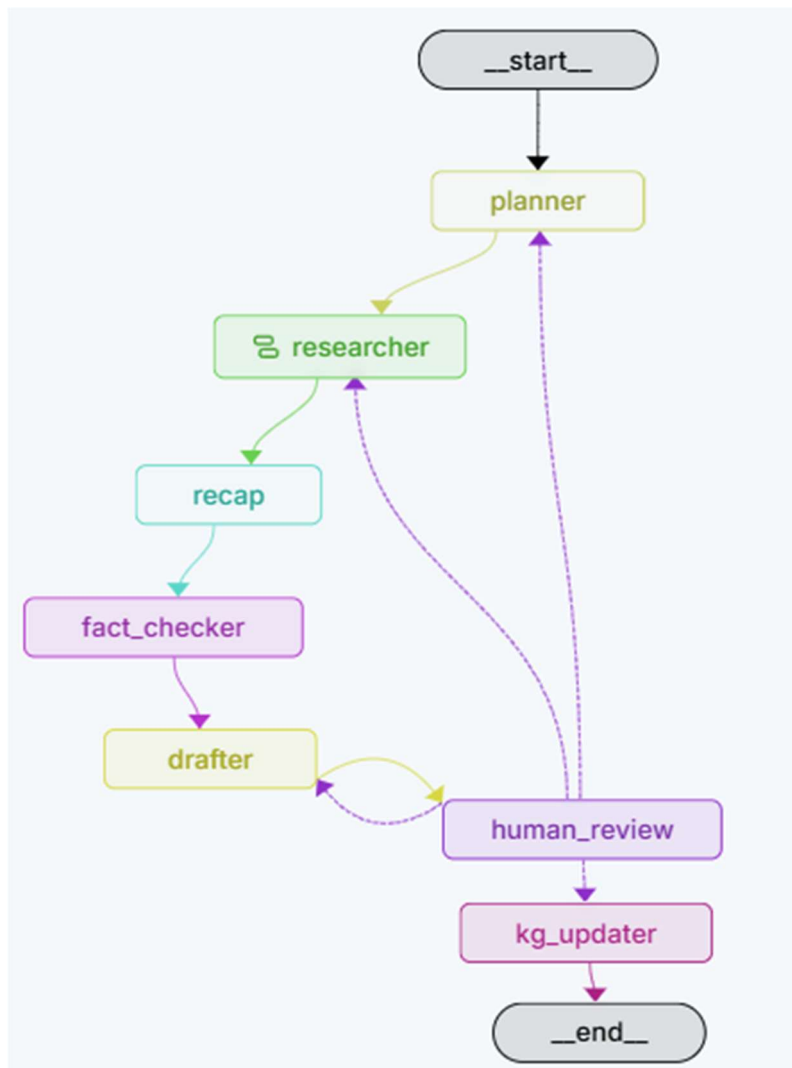
Infine, per ottemperare al requisito di Model Fine-Tuning e garantire l'assenza di allucinazioni nei dati generati, il sistema integra un modello specializzato nel fact-checking. Nello specifico, il modello base *mDeBERTa-v3-base-mnli-xnli* è stato sottoposto a fine-tuning mediante la tecnica LoRA (Low-Rank Adaptation), per valutare affermazioni sul dominio

turistico in classi di entailment, contradiction o neutral . Questo validatore automatico, combinato al meccanismo obbligatorio di *human-in-the-loop* per la revisione e l'approvazione finale, assicura che il Knowledge Graph venga popolato esclusivamente con informazioni accurate e verificate dall'utente.

2. Architettura

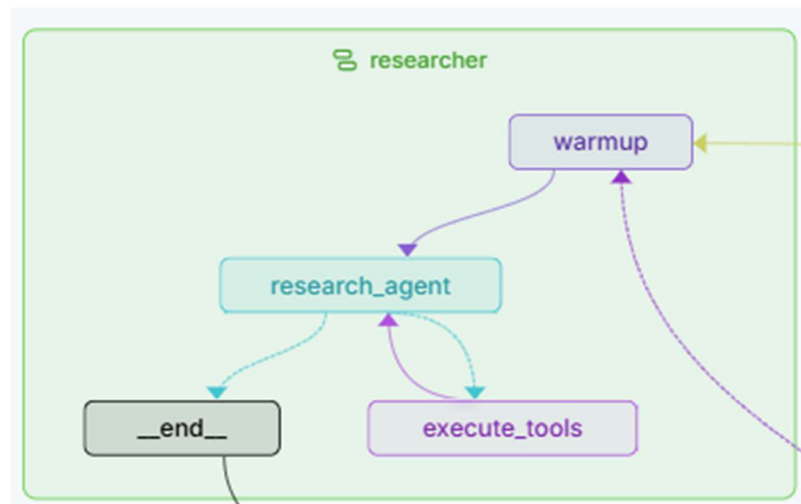
Il sistema è realizzato secondo un paradigma **multi-agente orchestrato a grafo di stato**, L'architettura del sistema "Copilot" è stata modellata e implementata come un grafo di stato direzionale (StateGraph) avvalendosi del framework LangGraph. Questo paradigma permette di orchestrare agenti e moduli logici definendo un flusso di esecuzione sequenziale e ciclico, governato da uno stato globale condiviso che mantiene traccia dell'intero processo editoriale. Ogni nodo svolge un ruolo preciso: pianificazione, ricerca, sintesi, verifica, scrittura, revisione e memorizzazione e comunica con gli altri esclusivamente attraverso un oggetto di stato condiviso che viene arricchito progressivamente lungo il percorso.

Questa organizzazione consente di combinare tre livelli complementari: un livello di orchestrazione (il grafo e i suoi nodi), un livello di conoscenza e memoria (il Knowledge Graph su Neo4j e il database vettoriale RAG), e un livello di strumenti (i tool con cui gli agenti interagiscono con fonti esterne, calcolano itinerari e verificano i fatti). Il modello linguistico principale utilizzato per il ragionamento e la generazione è llama-3.3-70b-versatile, servito tramite l'API Groq, mentre le decisioni di instradamento sfruttano l'output strutturato (structured output) validato da schemi Pydantic.



Il ciclo di vita della generazione di un articolo inizia dal nodo di partenza e attraversa una pipeline di elaborazione specializzata, fino a raggiungere il nodo terminale a seguito dell'approvazione e del salvataggio. Il grafo principale si articola nei seguenti nodi:

- **Planner:** Costituisce il punto di ingresso decisionale del sistema. Questo nodo interroga il Knowledge Graph per analizzare la copertura storica degli articoli e l'eventuale presenza di festival stagionali, valutando l'intento dell'utente per definire o aggiornare un calendario editoriale di tre articoli.
- **Researcher:** A causa della complessità delle operazioni di reperimento informazioni, questo modulo è implementato internamente come un sotto-grafo. Il suo comportamento segue un paradigma di ragionamento ReAct iterando ciclicamente tra una fase di analisi del contesto (warmup), la scelta autonoma delle azioni da parte dell'agente (research_agent) e l'esecuzione effettiva degli strumenti di ricerca K-RAG e web (execute_tools).



- **Recap:** Funge da modulo di sintesi strutturata. Estrae i dati grezzi raccolti dal Researcher e li organizza in un dossier formale in formato Markdown, garantendo che le informazioni logistiche, culturali e le info pratiche siano estrapolate in uno schema dati rigoroso e prive di duplicati.
- **Fact Checker:** Si occupa dell'estrazione delle affermazioni fattuali (claim) dal dossier generato. Tali affermazioni vengono successivamente sottoposte a un modello LLM specializzato e sottoposto a fine-tuning per classificarle metricamente in categorie di verità (Entailment, Contradiction, Neutral).
- **Drafter:** È il nodo deputato alla stesura creativa finale. Genera il corpo dell'articolo unendo il dossier di ricerca, il report di fact-checking e impaginando dinamicamente le immagini pertinenti recuperate attraverso l'interrogazione multimodale del modello CLIP.
- **Human Review:** Introduce un punto di interruzione obbligatorio (interrupt) per la supervisione. Il nodo mostra la bozza all'utente e, avvalendosi di un LLM per il routing semantico, classifica il feedback ricevuto reindirizzando dinamicamente il flusso verso il Planner (per un cambio radicale di meta), il Researcher (per approfondimenti), il Drafter (per modifiche stilistiche) o procedendo verso la pubblicazione.
- **KG Updater:** Rappresenta il nodo conclusivo della pipeline. Estrae le entità geografiche e tematiche definitive dall'articolo approvato, aggiorna le relazioni semantiche nel Knowledge Graph Neo4j, indicizza il documento testuale completo nel database vettoriale locale (ChromaDB) e infine aggiorna la coda del piano editoriale.

2.1 Gestione dello stato

Nel framework LangGraph, la comunicazione e la condivisione delle informazioni tra i vari nodi avvengono tramite uno stato globale mutabile, definito nel progetto dalla classe `BloggerState`. Questa struttura dati è implementata come un `TypedDict` di Python, all'interno del quale ogni chiave rappresenta una specifica variabile di contesto strettamente necessaria all'avanzamento della pipeline editoriale e al passaggio di informazioni tra gli agenti.

Di seguito vengono descritte nel dettaglio le variabili di stato che compongono il BloggerState:

- **messages:** Rappresenta la memoria conversazionale e cronologica del sistema. Grazie all'uso dell'annotazione e della funzione riduttrice `add_messages`, questa lista non viene sovrascritta a ogni iterazione, ma accumula in appendice i nuovi messaggi scambiati (es. messaggi di sistema, richieste dell'utente, invocazioni e risposte dei tool). È un elemento vitale per mantenere il contesto continuo durante l'esecuzione del loop di ragionamento (ReAct) dell'agente di ricerca.
- **user_input :** Memorizza la stringa di testo contenente il prompt iniziale o la richiesta di base fornita dall'utente in fase di avvio (ad esempio, l'input generico sulla meta da trattare).
- **kg_summary:** Questa variabile funge da ponte tra l'agente e il database a grafo. Mantiene i dati testuali e le sintesi estratte dinamicamente dal Knowledge Graph (come attrazioni vicine e storico delle coperture), fornendo agli agenti un bagaglio semantico pre-calcolato per guidare la stesura e la ricerca.
- **editorial_plan:** Contiene le informazioni specifiche (meta, formato del post e taglio editoriale) dell'articolo attualmente in fase di elaborazione nel ciclo corrente.
- **full_calendar:** Rappresenta la struttura completa del piano editoriale generato dal nodo Planner (generalmente composto da tre articoli). Mantenere la coda completa permette al sistema, a valle della pubblicazione, di scalare al post successivo aggiornando lo stato.
- **Action_result:** Un dizionario che aggrega i risultati grezzi restituiti dagli strumenti esterni. Ogni chiave corrisponde al nome del tool invocato (es. `scraping web` o interrogazione del vector DB), mappata a una lista contenente i log testuali di risposta generati durante la fase di reperimento delle informazioni.
- **reasoning_trace:** Struttura dati dedicata a garantire l'Explainable AI e a facilitare il debugging. Registra passo dopo passo la traccia cognitiva del sistema frammentando le operazioni nel formato agente, pensiero, azione e osservazione.
- **research_summary:** Contiene il dossier di ricerca finale e strutturato in formato Markdown. È il prodotto del nodo di Recap, che si occupa di distillare i dati caotici raccolti in `tool_outputs` organizzandoli in logistica, storia, cultura e info pratiche.
- **fact_check_report:** Raccoglie il report di validazione fattuale testuale prodotto dal modello LoRA specializzato. Fornisce al copywriter una "pagella" esplicita con i giudizi sulle singole affermazioni, indicando chiaramente quali dati siano attendibili (Entailment) e quali debbano essere scartati (Contradiction).
- **current_draft:** Memorizza la bozza testuale finale dell'articolo prodotta dall'agente Drafter. Questo testo include la formattazione Markdown, i contenuti generati e i segnaposti sostituiti dinamicamente con i link multimodali per le immagini.

- **human_feedback:** Variabile destinata a intercettare i commenti, le correzioni o le approvazioni fornite dal supervisore umano durante il blocco di Human-in-the-loop. Modifica dinamicamente lo stato guidando i successivi reindirizzamenti dell'esecuzione.

2.2 Nodo Planner

Il nodo Planner costituisce il punto di ingresso decisionale dell'architettura e ha il compito strategico di definire o gestire il calendario editoriale del blog. A differenza di un semplice generatore di testo, valuta lo storico preesistente e le intenzioni dell'utente prima di proporre una sequenza logica di pubblicazione.

Il funzionamento del nodo si articola in cinque fasi logiche fondamentali:

Recupero del Contesto

All'avvio, il nodo interroga il Knowledge Graph per estrarre informazioni cruciali atte allo sviluppo di una strategia editoriale data-driven. Nello specifico, il sistema recupera i gap di copertura geografica, i gap tematici, i festival stagionali relativi al mese corrente e la lista degli ultimi articoli pubblicati per prevenire contenuti ridondanti.

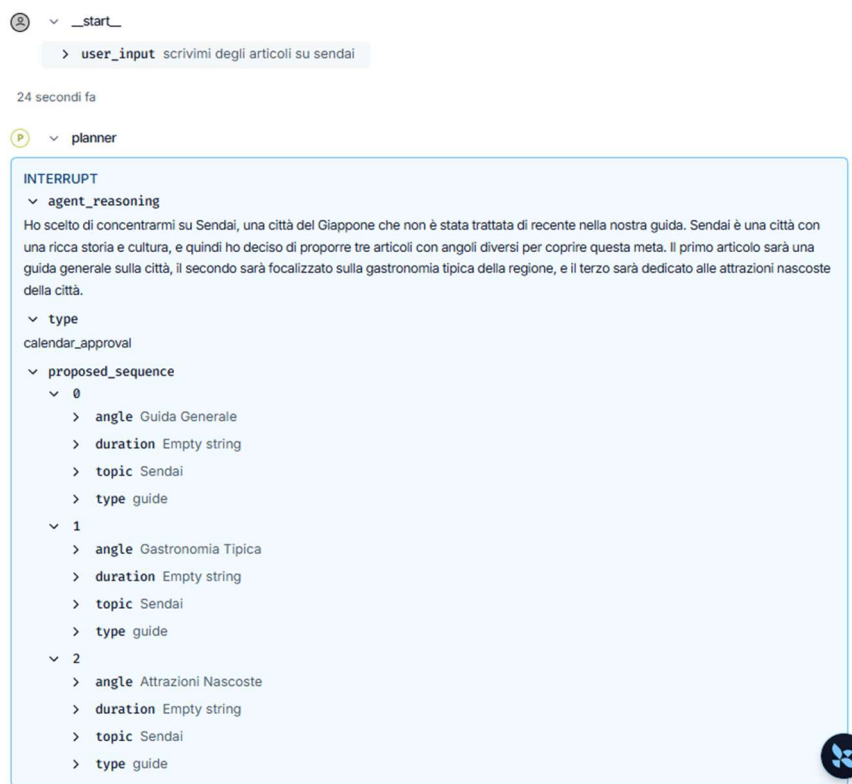
Analisi delle Intenzioni e Gestione degli Stati

Il sistema verifica innanzitutto la presenza di un piano editoriale in sospeso all'interno del database Neo4j. Se presente, il nodo si avvale di un modello LLM con output strutturato, vincolato allo schema PlannerIntent, per analizzare semanticamente l'input testuale dell'utente e classificarne l'intento logico. Di conseguenza, il nodo può operare in tre modalità principali:

- **Stato confirm_active:** Se l'utente acconsente a procedere con un piano precedentemente messo in pausa, il nodo salta completamente la fase di generazione. Seleziona la prima meta in coda dal database, resetta le variabili di stato per evitare interferenze e reindirizza il flusso direttamente alla fase di ricerca.
- **Stato modify_plan:** Se l'utente richiede variazioni a un piano già in coda, il sistema aggiorna il contesto includendo l'itinerario sospeso e istruisce l'LLM a rielaborare la sequenza tenendo conto del nuovo feedback.
- **Stato new_plan:** Se non vi sono piani attivi o se l'utente sottopone un prompt relativo a una meta completamente nuova, il sistema avvia la generazione di un calendario da zero.

La generazione vera e propria è delegata a un agente LLM che deve rispettare stringenti direttive architetturali e restituire i dati aderendo allo schema Pydantic EditorialCalendar. L'agente è istruito a proporre la sequenza rispettando regole matematiche e contenutistiche critiche:

- **Regola del Numero di Post:** La struttura decisionale è vincolata al tipo di richiesta. Se l'utente richiede un itinerario multi-tappa o una recensione specifica, l'agente deve generare esattamente 1 post. Se la richiesta è generica, l'agente deve generare tassativamente 3 post (guide) assegnando a ciascuno un focus editoriale (angle) completamente diverso e mai trattato per quella determinata città.
- **Gestione Duplicati Storici:** Attraverso il controllo dei post recenti estratti dal Knowledge Graph, qualora venga richiesta una città già trattata, l'agente è obbligato a variare drasticamente il taglio editoriale per garantire la freschezza dei contenuti.
- **Processo di Ragionamento:** L'LLM deve popolare un campo `reasoning_process` rivolgendosi direttamente all'utente per giustificare la selezione delle mete o l'eventuale variazione di focus dovuta a duplicati.



Una volta generata la proposta, il nodo entra in un loop di revisione sospendendo temporaneamente l'esecuzione del grafo tramite la funzione *interrupt* di LangGraph. L'utente visualizza le mete proposte e le giustificazioni logiche dell'agente, avendo la possibilità di fornire un feedback in linguaggio naturale. Tale feedback viene intercettato e analizzato da un ulteriore valutatore (basato sullo schema **PlanApprovalRouting**) che lo classifica in due categorie:

- Se **modify**: Il sistema concatena il feedback a uno storico interno e rigenera il piano.
- Se **approve**: Il ciclo si interrompe e il piano viene approvato definitivamente.

Superata la fase di interazione umana, il piano editoriale validato viene salvato in modo persistente nel database a grafo. Il nodo conclude la sua esecuzione estraendo la "meta del

giorno" (il primo elemento della sequenza) e istruendo un comando di update per il BloggerState.

2.3 Nodo Research

Il nodo researcher costituisce il motore di acquisizione e verifica delle informazioni dell'intera architettura. Per gestire la complessità delle operazioni e soddisfare i requisiti del paradigma di ragionamento strutturato (stile ReAct), questo componente è stato implementato tramite un `research_subgraph`. Il sottografo innesca un ciclo iterativo di esplorazione, valutazione e interrogazione dinamica degli strumenti. L'obiettivo è raccogliere dati fattuali e logistici, interrogare attivamente la memoria a lungo termine (Knowledge Graph) e sfruttare il retrieval ibrido sui documenti testuali (RAG), delegando le scelte operative a un agente autonomo. Il flusso si articola in tre nodi interni e un sistema di routing condizionale.

2.3.1 Nodo Warmup

Questo nodo rappresenta la fase di inizializzazione della ricerca.

- **Pre-caricamento K-RAG:** All'avvio, se non sono presenti feedback diretti o riassunti pregressi nello stato, il nodo esegue un'estrazione deterministica interrogando direttamente Neo4j tramite il `kg_query_tool`. Questo passaggio estrae il contesto iniziale relativo alla meta selezionata dal nodo Planner, operazione fondamentale per guidare le successive espansioni delle query e il ragionamento dell'agente.
- **Iniezione del System Prompt:** Il nodo costruisce e inietta il prompt di sistema per il modello linguistico. Una logica condizionale adatta dinamicamente le direttive in base al formato del post richiesto (itinerary, review o guide). Ad esempio, se il post è un itinerario, il prompt impone l'uso del `travel_route_tool` per calcolare distanze e tempistiche; se è una recensione, forza la ricerca su dati come prezzi esatti, pro/contro. Inoltre, le direttive stabiliscono i vincoli del comportamento ReAct: impongono la ricerca di dati pratici numerici, la spiegazione esplicita del ragionamento (Thought) prima di ogni chiamata e il divieto assoluto di query sature.
- **Inizializzazione dello Stato:** Aggiorna lo stato BloggerState inserendo il primo blocco narrativo nella `reasoning_trace` e impostando il contatore e le metriche di base per il tracciamento delle attività.

2.3.2 Nodo Agent

Ad ogni iterazione, analizza lo stato della ricerca per stabilire l'azione successiva.

- **Valutazione del Contesto:** Esamina la cronologia dei messaggi scambiati. Per ottimizzare la focalizzazione del modello, se l'ultimo messaggio in coda deriva dall'esecuzione di un tool, il nodo inietta temporaneamente un *ghost reminder* (un messaggio invisibile nell'output finale) che esorta l'LLM a valutare i risultati appena

ottenuti e a concludere la ricerca qualora i dati siano sufficienti a coprire il focus editoriale

- **Invocazione LLM:** Invoca il modello di base associato ai tool a disposizione. La risposta generata consisterà nella richiesta di invocazione di uno o più tool oppure nella generazione del testo discorsivo finale che interromperà il loop.

2.3.3 Nodo Execute Tools

Modulo che si occupa dell'esecuzione dell'azione (*Action*) determinata dal nodo Agent.

- **Esecuzione Dinamica:** Itera sulle chiamate agli strumenti richieste dall'LLM.
- **Tracciamento ReAct:** Cattura il pensiero dell'agente che ha motivato l'azione.
- **Aggiornamento del KG:** Nel caso l'agente decida di effettuare chiamate opzionali al `kg_query_tool` durante il loop, i nuovi frammenti informativi recuperati vengono direttamente aggiunti alla variabile di stato `kg_summary`. Gli output grezzi vengono infine collezionati nel dizionario `tool_outputs` e il contatore delle chiamate (`tool_call_count`) viene incrementato.

2.3.4 Routing Condizionale e Condizioni di Uscita (*route_research*)

Questo arco condizionale funge da controllore del ciclo.

- **Instradamento:** Valuta l'ultimo messaggio generato dal nodo Agent. Se contiene chiamate ai tool, instrada il grafo verso il nodo `execute_tools`, creando così il loop iterativo della ricerca.
- **Limite:** Per evitare loop infiniti, la funzione controlla il `tool_call_count` globale della sessione. Se l'agente raggiunge il limite prefissato, la ricerca viene interrotta forzatamente uscendo dal ciclo.
- **Terminazione Naturale:** Se l'agente ritiene di aver raggiunto il soddisfacimento dei criteri di ricerca e genera una risposta puramente testuale, il ciclo si conclude e il sottografo restituisce il controllo al flusso principale del sistema, passando la palla al nodo successivo.

2.4 Nodo Recap

Il primo passaggio consiste in un'operazione di pulizia e deduplicazione: il nodo itera su tutti gli output grezzi generati dai tool, scartando i frammenti di testo identici per ottimizzare la finestra di contesto da passare al modello linguistico. Contestualmente, tramite un'espressione regolare (regex), il nodo estrae preventivamente tutti gli URL effettivamente visitati dai tool, costruendo una *whitelist* di fonti reali e verificate (`allowed_urls`).

Successivamente, viene formulato un prompt di sistema che istruisce l'LLM a creare un riassunto. Le istruzioni del prompt variano drasticamente a seconda del formato del post

richiesto (itinerary, review o guide), garantendo che il modello si concentri sui dettagli rilevanti.

Per garantire il massimo rigore, l'utilizzo del metodo `with_structured_output` di LangChain instrada l'output verso tre differenti schemi Pydantic a seconda della tipologia di contenuto:

- **ResearchDossier:** Lo schema per le guide generiche, che organizza la ricerca in blocchi come storia, cultura, attrazioni principali e logistica.
- **ItineraryDossier:** Lo schema dedicato ai viaggi multi-tappa, che impone all'LLM di restituire una lista di oggetti `DayPlan` strutturati cronologicamente, focalizzandosi su tempi e logistica degli spostamenti.
- **ReviewDossier:** Lo schema per le recensioni, progettato per estrarre esclusivamente dettagli legati a una singola struttura.

Tutti i modelli ereditano da una struttura base (`BaseDossier`) e si appoggiano a sottomodelli modulari estremamente rigidi:

- **SourcedSection:** Separa rigidamente il contenuto testuale dall'elenco degli URL da cui il dato è stato estrapolato.
- **AttractionDetail:** Modella ogni singola attrazione turistica richiedendo attributi espliciti come name, description e un singolo `source_url` esatto, vietando allucinazioni.
- **PracticalInfoDetail:** Isola i dati ad altissimo rischio di errore (prezzi, orari di apertura, giorni di chiusura) ancorandoli a una fonte precisa.

Una volta che l'LLM ha istanziato correttamente l'oggetto JSON, il nodo esegue una compilazione deterministica via codice per tradurre la struttura in un documento Markdown. Anche l'impaginazione del Markdown è dinamica e si modella sui tre differenti tipi di dossier.

In questa fase interviene un rigoroso meccanismo anti-allucinazione definito **Citation Guard**: una funzione helper confronta e normalizza ogni singolo URL generato dall'LLM con la *whitelist* creata all'inizio del nodo. Qualora il modello produca un'allucinazione inventando un link, il sistema neutralizza la citazione forzando l'etichetta "nessuna", e traccia l'anomalia aggiungendo l'URL a una lista di scarti per l'observability. Attraverso un'ulteriore funzione dedicata (`format_sourced_section`), il sistema unisce in modo sicuro il testo discorsivo alle sue fonti verificate; qualora una sezione sia priva di fonti tracciate o sopravvissute al controllo, la funzione inietta forzatamente la stringa "[Fonte: Nessuna]".

In uscita, il nodo aggiorna lo stato globale popolandolo la variabile `research_summary` con il dossier formattato e aggiungendo un nuovo blocco alla `reasoning_trace` per documentare al sistema l'avvenuta estrazione isolata.

2.5 Nodo Fact checker

Lo scopo del nodo Fact checker è agire come un revisore editoriale indipendente, analizzando il dossier generato dal nodo recap per individuare, estrarre e validare matematicamente le affermazioni fattuali prima che queste vengano utilizzate per la stesura dell'articolo finale.

Il funzionamento del nodo si articola in due fasi distinte: l'estrazione semantica dei fatti e la validazione tramite inferenza in linguaggio naturale tramite un modello fine tuned.

Fase 1: Estrazione Strutturata dei Claim Fattuali

In presenza di un dossier valido, il sistema utilizza il modello linguistico di base **Llama-3.3-70b-versatile**. Il prompt di sistema forza l'LLM a ignorare opinioni o frasi poetiche, concentrandosi esclusivamente su fatti verificabili.

Lo schema ad oggetti si appoggia al sottomodello ClaimInfo, il quale impone l'estrazione di due campi fondamentali:

1. **claim:** L'affermazione fattuale isolata.
2. **paragrafo_contesto:** La porzione o frase esatta estratta *esclusivamente* dal database dei dati grezzi iniziali (output dei tool e dati storici del Knowledge Graph) che funge da prova inconfutabile per confermare o smentire il claim.

L'estrazione è limitata ai 4 claims più rilevanti.

Fase 2: Validazione tramite Modello Fine-Tuned (LoRA)

Una volta ottenuto l'array di ClaimInfo, il nodo abbandona l'utilizzo del LLM generalista e delega la valutazione a un modello specializzato tramite la chiamata alla funzione ***evaluate_claim_with_lora***.

Questa funzione si interfaccia con il modello pre-addestrato **mDeBERTa-v3-base-mnli-xnli**, precedentemente sottoposto a fine-tuning mediante tecnica LoRA (*Low-Rank Adaptation*) sul dominio turistico e sui task di fact-checking. Il modello non genera testo, ma esegue un compito di classificazione NLI (*Natural Language Inference*), valutando il claim rispetto al suo paragrafo_contesto.

Per ogni affermazione analizzata, il modello restituisce una delle tre seguenti classi probabilistiche:

- **ENTAILMENT:** Il claim è pienamente supportato dalle evidenze fornite nel contesto.
- **CONTRADDIZIONE:** Il claim è smentito o logicamente incompatibile con il contesto analizzato.
- **NEUTRO:** Il contesto non fornisce informazioni sufficienti per confermare o smentire il claim in modo definitivo.

Al termine del ciclo di validazione, il nodo formatta i risultati in un report leggibile denominato "Pagella del Fact-Checker Personale". Questo documento elenca, per ogni fatto analizzato, il testo del claim, il contesto di riferimento e il giudizio emesso dal modello LoRA.

Questo garantirà che il copywriter (l'LLM del nodo successivo) abbia a disposizione direttive esplicite su quali informazioni includere con sicurezza e quali scartare categoricamente.

2.6 Nodo Drafter

Il nodo `drafter_node` rappresenta il motore generativo e creativo dell'intera architettura. Il suo compito è sintetizzare la mole di dati elaborata dai nodi precedenti e redigere l'articolo finale, garantendo che il testo sia narrativamente coinvolgente, strutturalmente corretto e, soprattutto, rigorosamente aderente alle evidenze fattuali validate dal modello di fact-checking.

Questo nodo istanzia il modello linguistico (Llama-3.3-70b-versatile) con un parametro di temperature impostato a 0.2.

Viene utilizzato un *System Prompt* che impone regole editoriali ferree:

- **Gestione del Fact-Checking:** Il modello riceve istruzioni per allinearsi ciecamente alla "Pagella del Fact-Checker". Le informazioni etichettate come *CONTRADDIZIONE* o *Neutre* devono essere ignorate, mentre le evidenze classificate come *ENTAILMENT* possono essere integrate con sicurezza.
- **Meccanismo di Autovalutazione (Quality Control Loop):** Il nodo integra un ciclo di controllo qualità interno (con un massimo di 3 iterazioni) che convalida la bozza appena generata su parametri editoriali e strutturali predefiniti:
 - **Lunghezza minima:** L'articolo deve superare una soglia algoritmica di 500 parole per evitare testi superficiali o riassuntivi.
 - **Validazione della struttura:** Il nodo verifica programmaticamente, tramite espressioni regolari (regex), la presenza dei tag H1 (Titolo) e H2 (Sottotitoli). Controlla inoltre la presenza obbligatoria di una sezione logistica (es. "Come muoversi") qualora il dossier di partenza contenga effettivamente dati pratici su trasporti e spostamenti.
 - **Loop di correzione:** Se la bozza fallisce uno di questi controlli, il nodo innesca un ciclo di rigenerazione automatica passando gli errori rilevati come nuovo feedback restrittivo al modello, evitando di presentare bozze inadeguate alla fase di revisione umana.
- **Integrazione Multimodale (CLIP):** In questa fase, il nodo interroga le capacità multimodali del sistema. L'LLM inserisce segnaposti strategici nel testo ([IMAGE: descrizione]). Successivamente, il nodo invoca la funzione

`rag_manager.search_image`, che sfrutta gli embedding vettoriali generati dal modello `ProfCLIPEmbeddingFunction` per interrogare una collezione dedicata in ChromaDB (`travel_images`) e selezionare l'immagine la cui corrispondenza semantica è più alta rispetto alla descrizione richiesta. Per garantire la qualità visiva, un set di controllo viene aggiornato dinamicamente per escludere immagini già inserite ed evitare ridondanze.

In caso di feedback negativo proveniente dal nodo di revisione umana, il nodo `Drafter` altera dinamicamente il `System Prompt`, forzando il modello a riscrivere l'intero articolo obbedendo prioritariamente alle correzioni richieste, mantenendo però invariati i vincoli di fact-checking e impaginazione.

2.7 Nodo human feedback

Il nodo `human_in_the_loop_node` costituisce il "gatekeeper" dell'intera architettura. Questo nodo sfrutta le capacità di sospensione asincrona del framework `LangGraph` per integrare il giudizio dell'utente direttamente all'interno del processo decisionale dell'agente.

Il cuore meccanico del nodo risiede nell'invocazione della funzione `interrupt()` di `LangGraph`. Questa direttiva congela l'esecuzione del grafo di stato, salvandone il contesto in memoria, e si mette in attesa di un messaggio testuale contenente il feedback dell'utente.

Il sistema analizza il feedback dell'utente e lo passa ad un'istanza dell'LLM vincolata allo schema `Pydantic FeedbackRouting`.

Il *System Prompt* istruisce l'LLM a comportarsi come un router editoriale, classificando l'intento del revisore in una di quattro direttive operative e fornendo una giustificazione logica per la scelta:

1. **approve:** L'utente approva l'articolo e il flusso viene instradato verso il nodo finale `kg_updater`.
2. **rewrite:** L'utente richiede modifiche stilistiche, senza richiedere nuovi fatti. Il flusso viene retrocesso al nodo `drafter`.
3. **need_research:** L'utente richiede approfondimenti fattuali sulla meta attuale. Il flusso viene retrocesso all'inizio del sottografo di ricerca.
4. **change_topic:** L'utente rigetta la meta proposta e chiede un cambio radicale di argomento. Il flusso viene fatto ripartire dal principio, al nodo `planner`.

Un aspetto tecnicamente critico di questo nodo è la gestione della memoria del grafo durante i cicli di retrocessione. Per evitare che i vecchi frammenti di ragionamento o le vecchie chiamate ai tool contaminino i nuovi tentativi di generazione, il nodo esegue una scrupolosa pulizia dello stato.

2.8 Nodo kg updater

Il nodo kg updater rappresenta il terminale di persistenza e sincronizzazione dell'intera architettura agentica. Agisce esclusivamente sulle bozze che hanno superato il revisore umano, con il duplice obiettivo di aggiornare la memoria semantica a lungo termine (Knowledge Graph) e la memoria vettoriale (RAG).

Il funzionamento del nodo è un processo deterministico e sequenziale che si articola in quattro fasi distinte: l'estrazione delle entità, l'aggiornamento del grafo Neo4j, il popolamento di ChromaDB e la gestione della coda editoriale.

La prima operazione consiste nel processare il testo finale dell'articolo e il dossier di ricerca per estrapolare le entità strutturate. Per farlo, il nodo invoca un LLM vincolato allo schema Pydantic TravelMetadataExtractor. Il System Prompt istruisce il modello ad agire come un Data Engineer, imponendo regole critiche per risolvere le ambiguità geografiche e costruire una gerarchia spaziale rigorosa:

- **mapped_locations:** Invece di estrarre liste separate e sconnesse, il modello è costretto a istanziare una lista di oggetti LocationMapping. Per ogni luogo menzionato, l'LLM crea un collegamento inequivocabile tra l'attrazione (spot), la sua città esatta (city) e la sua prefettura di appartenenza (prefecture). Se l'articolo parla di una città in generale, compila i dati urbani lasciando lo spot vuoto, ignorando categoricamente i luoghi citati per puro paragone.
- Oltre alla rigida mappatura geografica, il modello estrae le categorie turistiche (category), cibi e artigianato menzionati (local_foods, crafts), gli URL delle fonti (sources) e i fatti verificati (claims).

Ottenuto il dizionario delle entità (extracted_entities), il nodo invoca il metodo update_after_approval del modulo kg_manager. Il database esegue le seguenti operazioni relazionali dinamiche tramite query Cypher:

- Crea o aggiorna il nodo centrale BlogPost, inserendo titolo, topic e timestamp (created_at o updated_at).
- Gestisce dinamicamente la gerarchia geografica leggendo l'array mapped_locations: crea le relazioni COVERS_PREFECTURE verso la prefettura, connette univocamente la città alla sua prefettura (IN_PREFECTURE) e ancora definitivamente lo spot alla sua città corretta (IN_CITY).
- Collega il post alle restanti entità tramite relazioni semantiche specifiche: ABOUT, COVERS_CITY, OF_CATEGORY, HAS_LOCAL_FOOD, HAS_CRAFT, USED_SOURCE e CLAIMS.
- Genera inferenze spaziali creando relazioni di vicinanza (NEAR) tra gli spot che condividono la stessa città o prefettura all'interno dell'articolo elaborato.

Parallelamente all'aggiornamento strutturato su Neo4j, il nodo si occupa della memoria testuale non strutturata. Il testo completo dell'articolo viene incapsulato all'interno di un oggetto Document di LangChain. A questo documento vengono allegati metadati cruciali per le future operazioni di filtraggio RAG: il titolo, il tipo di post, il focus editoriale, la prefettura principale e il tag source impostato esplicitamente a "blog_interno" per distinguerlo dal materiale del web.

3. Tool

All'interno del progetto sono stati implementati i seguenti tools: `Advanced_web_research`, `kg_query_tool`, `rag_retrieval_tool`, `travel_route_tool`, `fact_checking_tool`.

3.1 Advanced web research

Il tool `advanced_web_research` costituisce il principale motore di acquisizione dati esterno del sistema. Il suo obiettivo è ottenere guide turistiche e blog dal web, scartare i contenuti commerciali e inserire i dati filtrati nel database vettoriale locale.

Il processo di ricerca e validazione delle fonti è suddiviso in diverse fasi rigorose per garantire la qualità del contenuto:

- **Ricerca e Valutazione Iniziale (Tavily):** Il tool interroga il motore di ricerca Tavily richiedendo un massimo di quattro risultati per la query fornita dall'agente. Ogni risultato viene valutato tramite un punteggio di confidenza, scartando a priori i link con uno score inferiore alla soglia dello 0.35.
- **Blacklist Anti-OTA:** Per mantenere l'integrità editoriale, i domini appartenenti a Online Travel Agencies (come Booking, TripAdvisor ed Expedia) vengono intercettati e bloccati tramite una lista predefinita (`_banned_domains`).
- **Web Scraping Ibrido:** Il codice HTML delle pagine approvate viene scaricato e analizzato primariamente tramite la libreria `trafilatura` per estrarre il corpo del test e in caso di fallimento o restituzione di contenuto vuoto, applica un meccanismo di *fallback* utilizzando BeautifulSoup per garantire l'acquisizione del dato.
- **Analisi Anti-Spam e Validazione:** Il testo estratto viene analizzato dalla funzione `is_informative_travel_blog`, che scarta i contenuti troppo brevi (sotto i 400 caratteri) o che presentano un'elevata densità di terminologia commerciale (es. "voli low cost", "acquista il biglietto"). Una successiva funzione assicura che il testo non sia corrotto verificando la densità di caratteri speciali.

Una volta garantita la qualità del testo, il tool procede alla scomposizione e al salvataggio dei dati preparandoli per i nodi successivi:

- **Estrazione Immagini per CLIP:** Prima di processare il testo, il sistema estrae gli URL delle immagini presenti nella pagina originale. Le prime cinque immagini valide vengono inviate al gestore RAG, che le scarica e le vettorializza sfruttando il modello CLIP.
- **Chunking del Testo:** Il contenuto testuale viene diviso in frammenti semantici tramite il RecursiveCharacterTextSplitter, impostato con una dimensione fissa di 1000 caratteri e un overlap di 150 caratteri. Ogni singolo frammento generato viene prefissato con la stringa testuale della query originale (es. "META TURISTICA/QUERY: {query}"), al fine di non disperdere la traccia semantica dell'argomento durante le future operazioni di *retrieval* vettoriale.
- **Salvataggio e Limiti di Memoria:** I chunk generati vengono convertiti in oggetti Document, arricchiti con metadati essenziali (URL della fonte, titolo, ID del chunk) e salvati all'interno di ChromaDB. Per prevenire problemi di Out-Of-Memory, il tool impone un hard-limit di 1500 chunk massimi elaborabili per singola esecuzione.
- Conclusa la fase di indicizzazione nel database vettoriale locale, il tool esegue immediatamente una ricerca ibrida interrogando il RAG (rag_manager.search(query, k=8)). I documenti recuperati vengono formattati e restituiti direttamente all'LLM sotto forma di stringa, chiudendo l'operazione di scraping, archiviazione e lettura all'interno di una singola chiamata tool.

3.2 Kg query tool

Il tool kg_query_tool rappresenta l'interfaccia di interrogazione semantica del sistema, fondamentale per l'implementazione del paradigma K-RAG. Questo strumento permette all'agente di accedere alla memoria strutturata del blog ospitata su Neo4j, recuperando sia metadati spaziali sulle destinazioni turistiche, sia lo storico editoriale associato a specifiche località.

All'interno del kg_manager, la ricerca non si limita a un semplice matching testuale del nodo, ma implementa una logica di risoluzione avanzata (_detect_entity) progettata per risolvere le ambiguità semantiche. Il sistema cerca contemporaneamente su cinque etichette (Spot, City, Festival, Prefecture, Region) e, in caso di omonimia (ad esempio "Tokyo" come città o come prefettura), assegna il nodo corretto basandosi sull'esattezza della stringa e su un sistema di priorità matematico (dove Spot ha priorità massima 1 e Region priorità minima 5).

Una volta identificata la natura esatta dell'entità, il manager invoca query Cypher (_PROFILE_QUERIES) per mapparne le relazioni spaziali:

- **Spot:** Estrae le attrazioni vicine collegate dalla relazione: NEAR, le categorie di appartenenza (:OF_CATEGORY) e la localizzazione geografica completa (città, prefettura, regione), includendo eventuali festival di zona.

- **City:** Recupera le attrazioni situate al suo interno, i festival cittadini e la gerarchia regionale superiore.
- **Festival:** Isola il luogo esatto di svolgimento (città e prefettura) e le attrazioni limitrofe influenzate dall'evento.
- **Prefecture / Region:** Restituisce collegamenti macro-gerarchici, agglomerando città e prefetture subordinate.

Prima di restituire il payload, il tool esegue una funzione di deduplicazione (`_dedup`) su tutte le liste estratte per assicurare che array come "spots", "festivals" o "categories" non presentino ripetizioni che potrebbero alterare i calcoli di retrieval.

Infine, il tool formatta la risposta come una stringa JSON strutturata che include lo stato dell'operazione, il contesto topologico individuato (`krag_context`), i messaggi di errore e l'elenco esplicito delle eventuali entità non individuate nel grafo (`entities_not_found`). Questo output fornisce all'agente di ricerca l'esatto bagaglio di parole chiave, categorie e relazioni spaziali necessarie per espandere o raffinare dinamicamente le query successive destinate al database vettoriale, alimentando di fatto il motore K-RAG.

3.3 Rag retrieval tool

Il tool `rag_retrieval_tool` costituisce il punto di accesso primario dell'agente al database vettoriale locale. Progettato per recuperare descrizioni di attrazioni e informazioni storico-culturali sul Giappone, questo strumento delega l'elaborazione profonda della ricerca al modulo aggiornato `rag_manager`.

Per garantire la massima precisione nell'estrazione dei documenti, il sistema implementa una Ricerca Ibrida che combina due paradigmi complementari:

- **Ricerca Semantica Densificata:** Sfrutta gli embedding vettoriali generati dal modello *HuggingFace* (all-MiniLM-L6-v2) e archiviati in ChromaDB per catturare il significato intrinseco e concettuale della query.
- **Ricerca Lessicale (BM25):** Parallelamente, il sistema mantiene in memoria un indice basato sull'algoritmo testuale BM25. L'aggiornamento di questo indice viene gestito dinamicamente tramite un flag di stato e la funzione `_sync_bm25`, che lo sincronizza in modo "lightweight" ogniqualvolta vengano inseriti nuovi documenti nel RAG.

I risultati generati dai due approcci vengono fusi tramite un componente `EnsembleRetriever`, che assegna un peso equo (0.5 ciascuno) a entrambe le ricerche. In questo modo, l'agente beneficia sia della flessibilità semantica sia dell'esatta corrispondenza delle keyword.

Espansione Dinamica K-RAG: Logistica vs Tematica

Il vero cuore del sistema risiede nel metodo `krag_search`, che intercetta la query testuale e interroga il Knowledge Graph per individuare eventuali entità note menzionate dall'agente. Se vengono rilevate entità, il sistema biforca la strategia di espansione:

- **Modalità Logistica (`logistic_refine`):** Se la query originale contiene keyword operative (come "costo", "orario", "biglietto", "apertura"), il K-RAG disabilita l'espansione concettuale per evitare "rumore" e genera sotto-query atomiche mirate esclusivamente alla risoluzione di tali task per ogni singola entità identificata.
- **Modalità Tematica (`thematic_expand`):** In assenza di keyword logistiche, il sistema estrae dal grafo i termini di espansione correlati al seed (es. la prefettura di appartenenza, la regione o le categorie) e formula sotto-query arricchite, fondendo il nome dell'attrazione con il suo contesto topologico.

Re-Ranking e Filtro di Diversità

Tutti i documenti recuperati dalle varie sotto-query vengono raggruppati e deduplicati. Poiché l'aggregazione di più ricerche altera l'ordinamento per pertinenza originario, il sistema applica un passaggio di *Re-Ranking* matematico: sfrutta numpy per calcolare la Similarità del Coseno (Cosine Similarity) tra il vettore della query originale e le matrici vettoriali dei contenuti uniti, riordinando l'intero array in base al punteggio ottenuto.

Per ovviare alla problematica della Chunk Domination, la funzione search alla base estrae inizialmente un bacino di candidati maggiore e impone un limite rigido che ammette un massimo di due *chunk* provenienti dalla stessa sorgente.

Formattazione Dinamica dell'Output

A valle della selezione, i documenti superano una fase di formattazione gestita dal metodo `format_search_results`. Questa funzione traduce gli oggetti Document in una singola stringa testuale strutturata, adattando dinamicamente la formattazione in base all'origine del dato:

- **Fonti Web:** Espone il titolo della pagina web e l'URL sorgente originale (metadati `title` e `source`).
- **Archivio Blog Interno:** Riconosce i metadati generati dal nodo di update (`name`, `city`, `prefecture`) ed espone i dettagli topologici del vecchio articolo.

Infine, per prevenire il superamento della finestra di contesto, il contenuto effettivo di ciascun documento viene troncato in sicurezza ai primi 1800 caratteri. Il tool `rag_retrieval_tool` restituisce infine l'output preceduto da un *header* delimitato da `KRAG_GROUNDING_DELIM` che espone in chiaro la strategia K-RAG adottata, le entità riconosciute e le sotto-query eseguite, garantendo la piena *observability* del ragionamento per eventuali log in LangSmith.

3.4 Travel route tool

Il tool `travel_route_tool` fornisce all'agente le capacità di ragionamento spaziale e pianificazione logistica necessarie per redigere la sezione obbligatoria sui trasporti all'interno della bozza dell'articolo. Questo strumento colma il divario tra la generazione di testo e la fattibilità reale di un itinerario turistico, accettando in input una lista di località (es. ["Tokyo",

"Kyoto", "Osaka"] definita nello schema TravelRouteInput) e calcolando l'itinerario ottimizzato, le distanze e i tempi di percorrenza stimati.

Per ottimizzare le performance, ridurre i tempi di latenza e prevenire blocchi per *rate-limiting* da parte del provider esterno, la funzione di geocodifica (`geocode_location`) è ottimizzata in memoria tramite il decoratore `@lru_cache`. Memorizza i risultati delle chiamate a funzione in base agli argomenti passati. Se la funzione viene chiamata di nuovo con gli **stessi argomenti**, restituisce il risultato dalla cache invece di rieseguire il calcolo.

Una volta ottenute latitudine e longitudine per le località valide, il tool affronta il calcolo del percorso. Per farlo in modo efficiente, implementa un algoritmo euristico basato sulla logica del *Nearest Neighbor*. Il sistema fissa la prima città della lista come nodo di partenza; dopodiché, analizza iterativamente i nodi non ancora visitati per trovare quello a distanza minima. Il calcolo della distanza non è euclideo, ma sfrutta la formula dell'**Haversine** per tenere conto della curvatura terrestre. Infine, per approssimare in maniera realistica la discrepanza tra la distanza in linea d'aria e i reali percorsi ferroviari o stradali giapponesi, la distanza calcolata viene moltiplicata per una costante di correzione (1.15).

Basandosi sulle distanze ricalcolate per ogni tratta, il tool inferisce dinamicamente il mezzo di trasporto più congruo e stima i tempi di percorrenza, applicando delle soglie euristiche tarate sulle infrastrutture giapponesi:

- **Distanze superiori a 50 km:** Il sistema assegna il Treno ad alta velocità (Shinkansen), stimando una velocità di crociera di 250 km/h.
- **Distanze tra 20 e 50 km:** Il sistema consiglia il Treno espresso regionale, assumendo una velocità media di 100 km/h.
- **Distanze inferiori a 20 km:** La tratta viene classificata come percorso per Treno locale o Metropolitana, calcolata a 30 km/h.

Al termine delle computazioni, il tool non restituisce un testo discorsivo, ma compila e restituisce una stringa JSON strutturata. Questo payload include:

1. **L'itinerario ottimizzato:** Una rappresentazione visiva della sequenza corretta (es. "Tokyo → Kyoto → Osaka").
2. **La distanza totale in chilometri.**
3. **I dettagli delle tappe:** Un array contenente, per ogni spostamento, la località di partenza, quella d'arrivo, la distanza esatta, il mezzo consigliato e il tempo stimato in formato ore e minuti.

3.5 Fact checking tool

Il tool `fact_checking_tool` svolge un ruolo critico nel garantire l'affidabilità delle informazioni raccolte dal web, fungendo da strato di validazione anti-allucinazione. Invece di delegare la

validazione fattuale a un LLM generico, il tool sfrutta un modello specializzato locale per eseguire un task di *Natural Language Inference*.

Il cuore del tool è incapsulato nella funzione `evaluate_claim_with_lora`. Il modello di base selezionato per questo compito è ***MoritzLaurer/mDeBERTa-v3-base-mnli-xnli***, un modello della famiglia transformer nativamente pre-addestrato per la classificazione di sequenze multilingua. Su questa base, il sistema applica i pesi di un addestramento specifico effettuato tramite la tecnica parametrica efficiente LoRA (Low-Rank Adaptation), caricandoli dal percorso di rete locale `./model/fact_checker_lora` sfruttando la libreria `peft`.

Data la potenziale onerosità computazionale nel caricare ripetutamente un modello transformer in memoria (RAM o VRAM), il caricamento del tokenizer e del modello viene gestito in modalità *lazy* e salvato in stato globale (`_fc_model`, `_fc_tokenizer`): il caricamento avviene fisicamente solo alla prima invocazione del tool da parte dell'agente. Tutte le successive esecuzioni riutilizzano i tensori già istanziati, minimizzando i tempi di latenza dell'inferenza durante il loop di ricerca. Il modello viene inoltre esplicitamente configurato in modalità valutazione.

Quando l'agente individua un dato sensibile o ambiguo (es. orari, prezzi, date storiche), invoca il tool passando due parametri stringa fondamentali: il `context` (il paragrafo di riferimento estratto dai documenti) e il `claim` (l'affermazione esatta da verificare). Il processo di classificazione segue questi passaggi tecnici:

- **Tokenizzazione:** Il tokenizer prepara l'input unendo contesto e claim in un unico tensore; per evitare errori di *Out-Of-Memory* o il superamento dei limiti posizionali, applica un troncamento selettivo (`truncation="only_first"`) che riduce la lunghezza del contesto preservando intatto il claim, garantendo che la lunghezza massima non superi i 512 token.
- **Inferenza:** Il calcolo tensoriale viene avvolto in un blocco `torch.no_grad()` per inibire la costruzione del grafo computazionale e il calcolo dei gradienti, ottimizzando drasticamente l'utilizzo della memoria.
- **Estrazione Logit:** L'output grezzo della rete neurale (`outputs.logits`) viene processato con la funzione matematica `torch.argmax` lungo l'ultima dimensione (`dim=-1`) per identificare l'ID della classe con la probabilità predetta più alta.

Il risultato numerico viene mappato su un dizionario che restituisce all'agente uno dei tre giudizi formali previsti dai task NLI:

- **0: CONTRADDIZIONE** – L'affermazione diverge esplicitamente dai fatti presentati nel contesto.
- **1: ENTAILMENT** – L'affermazione è logicamente derivabile e pienamente supportata dal contesto.

- **2: NEUTRO** – Il contesto fornito non contiene dati adeguati per confermare o smentire il claim.

Il nodo Drafter sarà costretto a scartare le contraddizioni e i neutri.

4. Knowledge Graph

Il Knowledge Graph (KGraph) rappresenta la memoria a lungo termine e la base di conoscenza strutturata del sistema editoriale. È stato implementato utilizzando il database a grafo Neo4j, interrogato tramite Python. Il suo ruolo primario è duplice: da un lato supporta un approccio K-RAG (Knowledge Graph-augmented RAG) fornendo contesto extra alle ricerche documentali, dall'altro mantiene uno storico preciso delle attività editoriali per evitare la duplicazione dei contenuti e identificare i gap di copertura delle mete turistiche. Il grafo viene aggiornato dinamicamente in maniera incrementale dal nodo kg_updater solo dopo che una bozza ha ricevuto l'approvazione umana.

L'ontologia è stata progettata per mappare in modo rigoroso sia la gerarchia geografica giapponese sia i metadati editoriali. Per garantire l'integrità dei dati, quasi tutte le entità presentano constraint di unicità sui propri identificativi.

I nodi principali sviluppati sono:

- **Region:** Rappresenta le 8 macro-regioni del Giappone (es. Hokkaido, Kanto, Kansai). Presenta la proprietà univoca name.
- **Prefecture:** Rappresenta le 47 prefetture giapponesi. Anch'essa indicizzata con un vincolo di unicità sulla proprietà name.
- **City:** Sostituisce il concetto generico di municipalità, mappando le città esatte in cui sono collocate le attrazioni, in modo da non confondere i confini cittadini con quelli prefettizi. Ha una proprietà univoca name.
- **Spot:** Identifica i punti di interesse fisici e le attrazioni turistiche (es. templi, santuari, quartieri, parchi).
- **Category:** Definisce la natura o la tematica dell'itinerario e degli spot (es. Shrine, Temple, Modern/City, Nature/Hike, Food). Ha una proprietà univoca name.
- **Festival:** Rappresenta gli eventi periodici o i "matsuri" locali. Conserva come proprietà il name (univoco) e il month (mese) in cui si svolgono, un dato essenziale per pianificare articoli legati alla stagionalità.
- **LocalFood e Craft:** Entità generate dinamicamente per catalogare rispettivamente i cibi e l'artigianato locale estratti e menzionati negli articoli. Hanno un vincolo di unicità sulla proprietà name.
- **BlogPost:** È il nodo centrale dal punto di vista editoriale. Viene creato al momento dell'approvazione di un post e salva i metadati dell'articolo: title (univoco), type (es.

guide, review), base_topic, angle (il taglio editoriale), e i timestamp created_at e updated_at.

- **Source:** Registra gli URL esatti delle fonti web esterne utilizzate dal sistema per estrapolare le informazioni. Presenta un constraint di unicità sulla proprietà url.
- **Claim:** Memorizza le specifiche affermazioni fattuali isolate durante la fase di analisi del testo per consentire operazioni di fact-checking e validazione dei dati. Utilizza la proprietà text.
- **EditorialPlan:** Un nodo operativo speciale (con id fisso 'current_plan') adibito al salvataggio temporaneo in formato JSON (proprietà sequence) della coda di pubblicazione attiva decisa dal Planner.

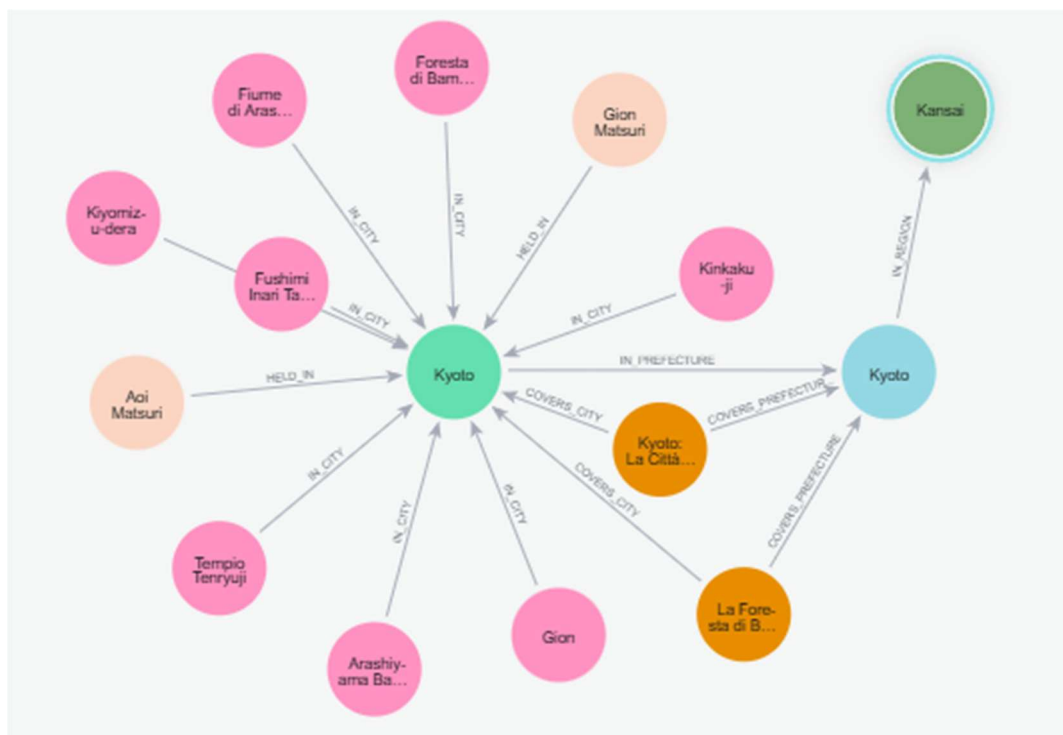
Le entità del grafo sono interconnesse tramite un set di relazioni direzionali che permettono al sistema di estrarre contesto semantico e inferire collegamenti.

Le relazioni sono strutturate come segue:

- **Connessioni Geografiche e Gerarchiche:**
 - (Prefecture)-[:IN_REGION]->(Region) collega ogni prefettura alla rispettiva macro-regione.
 - (City)-[:IN_PREFECTURE]->(Prefecture) inserisce le città nelle rispettive prefetture.
 - (Spot)-[:IN_CITY]->(City) collega l'attrazione turistica alla città in cui si trova.
- **Connessioni Turistiche e Relazionali:**
 - (Spot)-[:OF_CATEGORY]->(Category) associa una o più categorie tematiche ai punti di interesse.
 - (Spot)-[:NEAR]->(Spot) è una relazione bidirezionale di prossimità. Viene generata calcolando automaticamente quali spot condividono la stessa città o prefettura, essenziale per suggerire al modello attrazioni vicine durante la stesura (K-RAG).
 - (Festival)-[:HELD_IN]->(City) (o Prefecture) posiziona geograficamente gli eventi in calendario.
 - (Spot)-[:OFFERS_FOOD]->(LocalFood) lega un cibo locale allo specifico luogo turistico in cui viene offerto.
- **Connessioni Editoriali (Tracciabilità):** Il nodo BlogPost fa da ponte tra il contenuto testuale e il knowledge graph geografico/turistico.
 - (BlogPost)-[:ABOUT]->(Spot) traccia quali attrazioni specifiche sono le "protagoniste" dell'articolo.

- (BlogPost)-[:COVERS_CITY]->(City) e (BlogPost)-[:COVERS_PREFECTURE]->(Prefecture) mappano in modo esplicito quali zone geografiche sono state trattate dal post.
- (BlogPost)-[:OF_CATEGORY]->(Category), (BlogPost)-[:HAS_LOCAL_FOOD]->(LocalFood), e (BlogPost)-[:HAS_CRAFT]->(Craft) descrivono semanticamente gli elementi coperti dal testo.
- (BlogPost)-[:USED_SOURCE]->(Source) e (BlogPost)-[:CLAIMS]->(Claim) garantiscono la completa tracciabilità e "grounding" dell'articolo verso le fonti grezze del Web e i fatti analizzati dal tool di verifica LoRA.

L'implementazione sfrutta l'ontologia per alimentare la pipeline con il contesto pregresso. Per l'estrazione semantica, il sistema non si basa su tentativi a cascata, ma implementa la funzione `_detect_entity` che esegue un riconoscimento parallelo e pesato delle entità per risolverne le omonimie (es. Tokyo città vs Tokyo prefettura). Durante la fase K-RAG, il metodo `get_expansion_terms` interroga il grafo tipizzando i risultati in termini di "tema" (categorie, regioni) ed "entità" (spot vicini). Questo fornisce al motore ibrido di ricerca i termini esatti per espandere le query verso il Vector Database. Infine, al momento dell'approvazione del Direttore, un LLM strutturato estrae le entità testuali dalla bozza e passa i dizionari al metodo `update_after_approval` per creare i nodi mancanti tramite query MERGE di Cypher



5. RAG

Il modulo RAG è il motore di elaborazione e recupero delle informazioni non strutturate (testi e immagini), agendo in sintonia con il Knowledge Graph per formare un sistema K-RAG. Il cuore dell'infrastruttura di storage è basato su ChromaDB, configurato per mantenere la persistenza

locale dei dati in specifiche directory. Per gestire le diverse tipologie di informazioni, il database è stato architettato per gestire collezioni multiple, separando lo storage testuale da una collezione dedicata esclusivamente alle immagini.

Modelli di Embedding e Ricerca Ibrida (Hybrid Search)

Per massimizzare l'accuratezza del recupero delle informazioni testuali, il sistema non si affida a un'unica tecnica di retrieval.

- **Embedding Semantico:** I testi vengono vettorializzati utilizzando il modello all-MiniLM-L6-v2 di HuggingFace, ideale per catturare il significato semantico e concettuale dei documenti.
- **Ricerca Lessicale (BM25):** Parallelamente, il sistema mantiene in memoria un indice basato sull'algoritmo BM25 (BM25Retriever), che garantisce il recupero basato sulle frequenze esatte delle parole chiave. L'indice viene sincronizzato dinamicamente ogni volta che il database subisce variazioni.
- **Ensemble Retriever:** Le due tecniche vengono unificate tramite l'EnsembleRetriever di LangChain, assegnando un peso del 50% alla ricerca semantica vettoriale e un peso del 50% alla ricerca lessicale BM25. Questo approccio garantisce che le query dell'agente trovino sia documenti concettualmente affini sia documenti contenenti entità turistiche dai nomi molto specifici.

Espansione Dinamica K-RAG e Re-Ranking Vettoriale

Il metodo `krag_search` utilizza attivamente la struttura del Knowledge Graph per manipolare la query di ricerca prima dell'estrazione. Il sistema individua quali entità del KG sono menzionate nella query e adotta due strategie di espansione:

- **Raffinamento Logistico:** Se la query contiene keyword operative (come "costo", "prezzo", "biglietto", "orari", "apertura"), il K-RAG non applica l'espansione tematica. Il sistema segmenta la query nelle sue entità principali e genera sotto-query atomiche focalizzate esclusivamente sulla logistica, preservando le keyword mirate senza introdurre ulteriori elementi.
- **Espansione Tematica:** In assenza di keyword logistiche, il sistema invoca il Knowledge Graph per estrarre i termini topologici vicini al seed (come categorie, regione, prefettura). Questi termini vengono concatenati al seed originale per generare sotto-query arricchite e diversificate (evitando ripetizioni testuali).

Poiché la fusione di documenti provenienti da diverse sotto-query altera l'ordinamento nativo per pertinenza, il modulo esegue un **Re-Ranking matematico**: sfruttando la libreria numpy, il sistema calcola la similarità del coseno tra il vettore della query originale e le matrici vettoriali dei contenuti fusi, riordinando l'array finale in base a questo punteggio matematico.

Mitigazione della "Chunk Domination" e Filtraggio delle Fonti

Una problematica comune nei sistemi RAG è la tendenza a recuperare frammenti di testo (chunk) appartenenti tutti allo stesso documento originale, limitando la pluralità di informazioni. Per ovviare a questo problema, la funzione `search` del `rag_manager` implementa un filtro di diversità logico sui risultati restituiti dall'`EnsembleRetriever`. Durante la raccolta, il sistema tiene traccia dei metadati della sorgente e applica un limite massimo di chunk prelevabili per ogni URL. In questo modo si impone all'LLM di leggere fonti eterogenee.

Processo di Ingestion Dinamica e Segmentazione Testuale

A differenza dei database vettoriali tradizionali pre-popolati, il RAG progettato si arricchisce dinamicamente durante il flusso operativo:

- **Web Scraping:** Le pagine web scaricate vengono processate e suddivise tramite un `RecursiveCharacterTextSplitter` con una dimensione di 1000 caratteri e un overlap di 150, per non spezzare il contesto narrativo. Questi documenti vengono immediatamente inseriti nel Vector DB locale.
- **Archiviazione dello Storico Editoriale:** Quando il nodo `kg_updater` registra un articolo approvato, contestualmente ne salva il testo completo all'interno di ChromaDB, etichettandolo con metadati spaziali (es. città, prefettura) e categorizzandolo come "blog_interno".

Funzionalità Multimodale: Gestione Visiva tramite CLIP

Un aspetto avanzato dell'architettura RAG è la sua estensione al dominio visivo. Il sistema integra il modello vision-language `openai/clip-vit-base-patch32` di Hugging Face attraverso una classe custom `ProfCLIPEmbeddingFunction`, che esegue l'embedding congiunto sia su stringhe testuali sia su tensori di immagini. Il ciclo di vita delle immagini nel RAG avviene in due fasi:

1. **Estrazione:** Durante il web scraping, gli URL delle immagini vengono analizzati; quelle con risoluzione idonea (superiori a 400x300 pixel) vengono scaricate, convertite in matrici NumPy ed elaborate da CLIP per essere salvate nella collection di ChromaDB dedicata (`travel_images`).
2. **Immissione Dinamica:** Durante la stesura del post, l'LLM inserisce speciali segnaposti testuali. Utilizzando la funzione `search_image`, il sistema esegue una query testuale sullo spazio vettoriale di CLIP e restituisce l'URL dell'immagine semanticamente più vicina, scartando contestualmente i doppietti visivi (`exclude_urls`).

6. Fine Tuning

L'agente editoriale, nella sua fase di ricerca, raccoglie informazioni da fonti eterogenee (ricerca web, RAG e Knowledge Graph) e le condensa nella variabile di stato `research_summary`; il nodo `Drafter` le trasforma poi in un articolo di guida turistica. Questa

architettura, come qualunque pipeline basata su modelli generativi, è esposta al rischio di *allucinazioni* e, più in generale, alla produzione di affermazioni non supportate dalle fonti effettivamente recuperate. Nel dominio specifico del progetto l'errore non è astratto: prezzi dei biglietti, orari di apertura, durate dei trasporti, regole di etichetta e date stagionali sono dati puntuali e verificabili, e una guida che riporta un orario o un prezzo sbagliato produce un danno concreto alla credibilità del prodotto e all'esperienza del lettore. Serve dunque un meccanismo di controllo automatico, deterministico e a basso costo, capace di dire, per ogni affermazione fattuale, se essa sia effettivamente sostenuta dal materiale di partenza.

Data una coppia costituita da una *premissa* (il paragrafo-fonte, cioè il contesto realmente recuperato) e da un'*ipotesi* (il claim da verificare), il modello deve assegnare una tra tre etichette:

- **ENTAILMENT (1)** — il contesto conferma l'affermazione;
- **CONTRADICTION (0)** — il contesto smentisce l'affermazione;
- **NEUTRAL (2)** — il contesto non contiene informazioni sufficienti per confermare o smentire.

Il modello si colloca nel nodo `fact_checking_node`: un LLM estrae dal research summary fino a 4 claim fattuali, ciascuno accompagnato dal proprio paragrafo di contesto esatto; ogni coppia (contesto, claim) viene quindi sottoposta al modello fine-tuned e i verdetti vengono assemblati nel `fact_check_report`. Tale report viene infine passato al Drafter, che è vincolato a rispettarlo nella stesura definitiva. Il fact-checker agisce quindi come un vero e proprio *filtro di qualità* tra ricerca e scrittura.

Baseline

La baseline scelta è il modello di base MoritzLaurer/mDeBERTa-v3-base-mnli-xnli usato in modalità zero-shot, cioè senza alcun addestramento sul nostro dominio, e valutato sullo stesso identico test set adoperato per il modello fine-tuned.

La baseline viene calcolata istanziando una **pipeline di text-classification** sul modello pre-addestrato; per ogni esempio del test set si costruisce l'input concatenando premessa e ipotesi separate dal token SEP, si ottiene la predizione NLI nativa del modello e la si rimappa sullo schema di etichette del progetto (l'etichetta che contiene «entailment» viene mappata su 1, quella che contiene «contradiction» su 0, ogni altro caso su 2, cioè neutral). Sulle predizioni così ottenute si calcolano le stesse metriche usate per il modello finale.

La valutazione di questa baseline sul Test Set ha registrato un'Accuracy dello **0.8980 (89.80%)** e un Macro-F1 dello **0.8945 (89.45%)**. Seppur già validi, questi risultati rappresentano il punto di partenza da superare attraverso il fine-tuning con LoRA.

Metriche di Valutazione Utilizzate

La valutazione si fonda su due metriche principali, **Accuracy** e **Macro-F1**.

L'**Accuracy** è la frazione di coppie (contesto, claim) classificate correttamente. È intuitiva e immediata, ma da sola insufficiente per questo compito. Le classi non sono perfettamente bilanciate e, soprattutto, i tre tipi di errore non sono ugualmente accettabili.

Il **Macro-F1** è quindi la metrica primaria del progetto. L'F1 di una classe è la media armonica tra *precisione* e *richiamo* su quella classe; la versione *macro* ne calcola la media aritmetica non pesata sulle tre classi.

La funzione di perdita usata in addestramento è una *cross-entropy pesata* (WeightedTrainer): i pesi delle classi sono calcolati sul training set come inverso della frequenza, secondo la formula $\text{peso_c} = N / (\text{num_classi} \cdot \text{conteggio_c})$. Non si tratta di una metrica di valutazione, ma di una contromisura allo sbilanciamento, poiché impedisce che la classe più frequente domini l'aggiornamento dei pesi.

Struttura e Generazione del Dataset

Per addestrare il modello a comprendere le logiche di fact-checking specifiche del dominio turistico giapponese, è stato sviluppato un dataset NLI in lingua italiana in modo programmatico e strettamente controllato. Lo script `generate_factcheck_dataset.py` automatizza la creazione del dataset a partire da una base di fatti reali.

- **Composizione del Fatto:** Ogni elemento della base dati originale contiene un identificativo univoco (id), due diverse parafrasi testuali che fungono da contesto di riferimento (contexts), e liste separate di claim classificate in base alla loro veridicità rispetto al contesto: entail, contradict e neutral.
- **Formato in Uscita:** Lo script espande permutando ogni contesto con tutti i suoi claim associati, generando un file CSV3 (`factcheck_dataset_japan.csv`) in cui ogni riga presenta: fact_id (l'ID del fatto sorgente), context (la premessa), claim (l'ipotesi) e la label numerica (0, 1 o 2).

Suddivisione e Bilanciamento del Dataset

Il dataset generato conta complessivamente **979 campioni** derivati da 30 gruppi di fatti distinti. La divisione viene effettuata in modo sequenziale sulle righe rimescolate tramite le funzionalità native di `Dataset.train_test_split` di Hugging Face.

La partizione del dataset è stata impostata con il seguente rapporto: 70% per l'addestramento, 15% per la validazione e 15% per il test finale isolato.

La distribuzione numerica esatta dei campioni è la seguente:

- **Train Set:** 685 campioni totali (211 Contradiction, 216 Entailment, 258 Neutral/Hard).
- **Validation Set:** 147 campioni totali (44 Contradiction, 46 Entailment, 57 Neutral/Hard).

- **Test Set:** 147 campioni totali (45 Contradiction, 40 Entailment, 62 Neutral/Hard).

Nonostante un leggero sbilanciamento verso la classe, il `WeightedTrainer` custom implementato compensa calcolando e applicando dinamicamente pesi specifici alla funzione di Loss in base alle reali frequenze del Training Set.

Da questa distribuzione del training set vengono infine derivati i pesi delle classi impiegati nella cross-entropy pesata, con $N = 685$ esempi di addestramento si ottengono i valori $[1,082; 1,057; 0,885]$ rispettivamente per contraddizione, entailment e neutrale: la classe più numerosa riceve un peso inferiore a 1 e le due meno numerose un peso leggermente superiore, così da compensare lo sbilanciamento residuo durante l'addestramento.

6.5 La strategia di fine-tuning

Il punto di partenza è **MoritzLaurer/mDeBERTa-v3-base-mnli-xnli**: un modello *encoder* multilingue della famiglia DeBERTa-v3-base già specializzato sul compito di Natural Language Inference.

Adattare l'intero modello avrebbe significato aggiornare tutti i suoi parametri nell'ordine delle centinaia di milioni. Su un dataset di dimensioni contenute come il nostro (685 esempi di training) questo approccio è sconsigliabile per due ragioni: da un lato è costoso in memoria e tempo di calcolo, dall'altro espone a un elevato rischio di overfitting e di catastrophic forgetting, cioè la perdita delle capacità di ragionamento inferenziale che il modello ha acquisito nel pre-addestramento.

Abbiamo perciò scelto un approccio **PEFT (Parameter-Efficient Fine-Tuning)** basato su **LoRA (Low-Rank Adaptation)**. L'idea di LoRA è di congelare i pesi originali del modello e di iniettare, accanto ad alcune matrici di peso, una coppia di piccole matrici a rango ridotto il cui prodotto rappresenta l'*aggiornamento* appreso. Solo queste matrici vengono addestrate: il modello di base resta intatto e viene semplicemente "corretto" da un adattatore leggero. Nel nostro caso i parametri effettivamente addestrati sono **444.675**, ovvero circa lo **0,16% del modello complessivo**.

Per specializzarlo sul task del progetto, è stata istanziata una testa di classificazione specifica tramite la classe `AutoModelForSequenceClassification` impostata su 3 etichette in uscita, corrispondenti alle classi: *contradiction*, *entailment* e *neutral*.

In sintesi, l'addestramento agisce su due componenti: gli adattatori LoRA a basso rango nei layer di attenzione e la testa di classificazione, riaddestrata per intero.

Configurazione

La configurazione degli adattatori è definita dai seguenti parametri:

- **Rank.** È la dimensione interna delle matrici a rango ridotto e regola la "capacità" dell'adattatore: un rank più alto consente di apprendere correzioni più espressive ma

aggiunge parametri e aumenta il rischio di overfitting. Il valore del rank non è stato fissato a priori ma scelto tramite ricerca tra i candidati 8 e 16;

- **Alpha.** È un fattore di scala che moltiplica il contributo dell'adattatore prima di sommarlo al peso originale, secondo il rapporto α/r . È stato posto a 16: con $r = 16$ il fattore di scala risulta pari a 1, cioè l'aggiornamento a basso rango viene sommato senza amplificazione né attenuazione.
- **Dropout.** È una tecnica di regolarizzazione che, durante l'addestramento, azzerava casualmente una frazione degli input dell'adattatore, riducendo la co-dipendenza tra i parametri e quindi l'overfitting. Nel notebook è impostato a 0,1. È una scelta prudente data la piccola dimensione del dataset, dove il controllo dell'overfitting è prioritario.
- **Target Modules:** Ci concentriamo sui meccanismi di attenzione ["query_proj", "key_proj", "value_proj"].
- **Bias.** Il parametro bias è stato impostato su "none", mantenendo invariati i bias originali del modello per preservare la stabilità.
- **Tipo di task.** Il task_type è impostato a SEQ_CLS (sequence classification), affinché la libreria PEFT configuri correttamente l'architettura per la classificazione di sequenze.

Hyperparameters

Gli iperparametri del ciclo di ottimizzazione sono stati scelti come segue.

- **Learning rate.** Anch'esso oggetto di ricerca sistematica tra i valori $1e-4$, $2e-4$ e $3e-4$. È un tasso di apprendimento relativamente elevato, ma appropriato per il fine-tuning con LoRA: dovendo addestrare pochissimi parametri (le sole matrici dell'adattatore e la testa) partendo da inizializzazioni prossime allo zero, un learning rate più alto rispetto ai tipici valori del full fine-tuning (dell'ordine di $2e-5$) accelera la convergenza senza destabilizzare il modello di base, che resta congelato.
- **Batch size.** Fissata a 8 esempi per dispositivo, sia in addestramento sia in valutazione.
- **Numero di epoche ed early stopping.** L'addestramento finale è configurato per un massimo di 15 epoche, ma è governato da un meccanismo di early stopping con pazienza 3: se il Macro-F1 sul validation set non migliora per tre epoche consecutive, l'addestramento si interrompe.
- **Weight decay.** Impostato a 0,01, aggiunge alla funzione di perdita una penalità proporzionale alla norma dei pesi.
- **Warmup.** Con warmup_ratio = 0,1, il learning rate cresce linearmente da zero al valore nominale durante il primo 10% dei passi, per poi decrescere. Questa fase di riscaldamento stabilizza le prime iterazioni, quando i gradienti degli adattatori appena inizializzati possono essere rumorosi.
- **Ottimizzatore.** Viene usato l'ottimizzatore predefinito del Trainer, AdamW, con lo scheduler lineare del learning rate coerente con il warmup descritto.
- **Funzione di perdita pesata.** L'addestramento non usa la cross-entropy standard ma una sua versione pesata, implementata estendendo il Trainer in una classe WeightedTrainer che sovrascrive il metodo compute_loss. I pesi delle classi vengono

passati alla `cross_entropy`, così che gli errori sulle classi meno rappresentate pesino di più.

- **Tokenizzazione.** Le coppie (premessa, ipotesi) vengono tokenizzate insieme, con troncamento a una lunghezza massima di 256 token in addestramento.

I valori di rank e learning rate non sono stati scelti per intuizione ma tramite una ricerca a griglia (grid search) sulle quattro combinazioni possibili di $r \in \{8, 16\}$ e $lr \in \{1e-4, 2e-4, 3e-4\}$. Per ciascuna combinazione è stato addestrato un modello e ne è stato misurato il Macro-F1 sul validation set.

Rank	LR	Epoca fermata	Accuracy	Macro f1
8	0.0003	4.0	0.979592	0.978245
16	0.0002	6.0	0.979592	0.977753
16	0.0003	6.0	0.972789	0.971146
8	0.0001	6.0	0.972789	0.970845
16	0.0001	8.0	0.972789	0.970845
8	0.0002	6.0	0.972789	0.970312

La configurazione vincente è quella con $r = 8$ e $lr = 3e-4$, che raggiunge il Macro-F1 più alto sul validation set. Questa configurazione è stata quindi adottata per l'addestramento finale.

6.7 Confronto con la baseline e analisi dei risultati

6.7.1 Confronto quantitativo con la baseline

Il modello ottenuto tramite fine-tuning viene valutato sul *test set* tenuto separato per tutta la procedura e messo a confronto con la baseline zero-shot, usando le due metriche definite in precedenza: Accuracy e Macro-F1. La tabella seguente riassume l'esito.

Modello	Accuracy	Macro-F1
Baseline: Zero-Shot NLI (mDeBERTa-v3-base-mnli-xnli)	0,8980	0,8945

Modello	Accuracy	Macro-F1
Modello fine-tuned (LoRA, $r=8$, $lr=3e-4$)	0,9796	0,9777

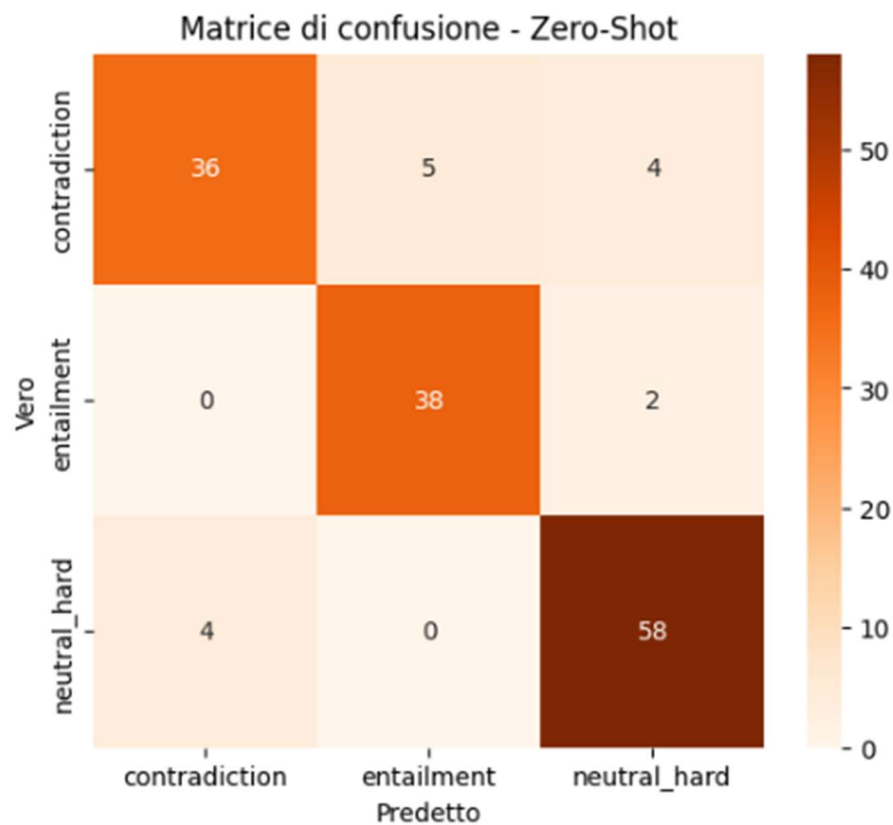
È importante sottolineare che la baseline *non era un avversario debole*: con un Macro-F1 di 0,89 il modello pre-addestrato dimostrava già una solida capacità di ragionamento inferenziale generico.

```

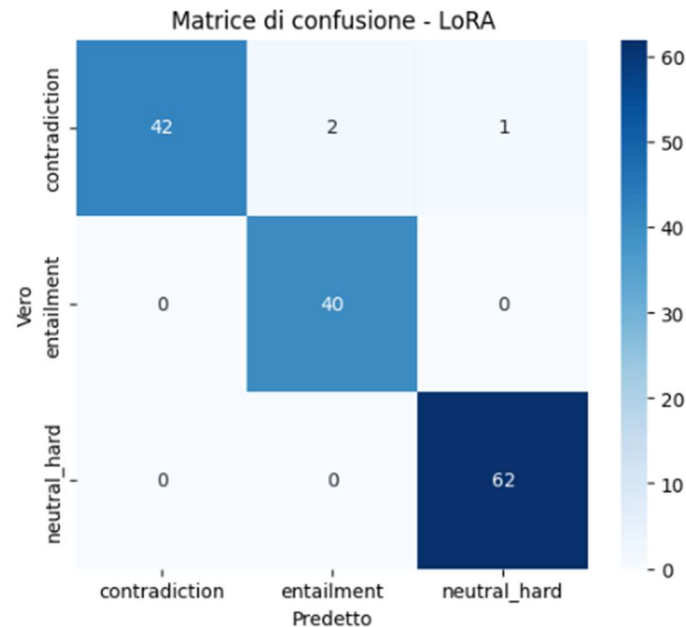
Classification Report per classe (Zero-Shot):
              precision    recall  f1-score   support

contradiction      0.90      0.80      0.85         45
  entailment       0.88      0.95      0.92         40
neutral_hard       0.91      0.94      0.92         62

```



Classification Report per classe (LoRA):				
	precision	recall	f1-score	support
contradiction	1.00	0.93	0.97	45
entailment	0.95	1.00	0.98	40
neutral_hard	0.98	1.00	0.99	62



La rilevanza del miglioramento risulta ancora più evidente ragionando in termini di **errori assoluti** sui 147 esempi del test set. La baseline ne classificava correttamente circa 132, sbagliandone **15**; il modello fine-tuned ne azzecca **144**, con soli **3 errori**. Il numero di errori si riduce dunque da 15 a 2

Alcuni esempi classificati correttamente:

```

--- SUCCESS ANALYSIS (Esempi classificati correttamente) ---
-----
CONTESTO (Premise): A Tokyo il mercato esterno di Tsukiji vive di mattina: le bancarelle iniziano ad aprire verso le 5:00 e la maggior parte delle attività abbassa le serrande nel primo pomeriggio, attorno alle 14:00.
CLAIM (Hypothesis): Il mercato esterno di Tsukiji apre solo la sera.
ETICHETTA VERA: contradiction | PREDETTO: contradiction (confidenza: 99.8%)
-----
CONTESTO (Premise): Mangiare in Giappone può essere economico: un pasto in una catena di gyudon come Yoshin oya o Sukiya costa circa 500-700 yen, mentre un ramen in un locale medio si aggira sui 1000 yen.
CLAIM (Hypothesis): In Giappone si può mangiare in modo economico nelle catene di gyudon.
ETICHETTA VERA: entailment | PREDETTO: entailment (confidenza: 100.0%)
-----
CONTESTO (Premise): A Gion, il quartiere storico di Kyoto, dal 2024 alcuni vicoli privati sono off-limits per i turisti, con multe fino a 10000 yen per i trasgressori; le vie pubbliche principali, come Hanamikoji, restano percorribili.
CLAIM (Hypothesis): A Gion si trovano numerose case da tè storiche chiamate ochaya.
ETICHETTA VERA: neutral_hard | PREDETTO: neutral_hard (confidenza: 99.9%)
-----
Totale corretti sul test: 144/147 (98.0%)

```

Esempio di errore:

--- ERROR ANALYSIS (Analisi di 3 Fallimenti) ---

CONTESTO (Premise): Nel quartiere di Gion a Kyoto, dal 2024 è vietato ai turisti entrare in alcuni vicoli privati; chi viola il divieto rischia una multa di 10000 yen. Le strade pubbliche principali come Hanamikoji restano accessibili.

CLAIM (Hypothesis): Dal 2024 l'intero quartiere di Gion è chiuso ai visitatori.

ETICHETTA VERA: contradiction | PREDETTO DAL MODELLO: entailment

CONTESTO (Premise): Lasciare la mancia in Giappone non è consuetudine e nei ristoranti può addirittura mettere in imbarazzo il personale. Di norma si paga alla cassa e non al tavolo.

CLAIM (Hypothesis): I camerieri giapponesi si aspettano una mancia generosa.

ETICHETTA VERA: contradiction | PREDETTO DAL MODELLO: neutral_hard

CONTESTO (Premise): Shibuya Sky è la terrazza panoramica in cima allo Shibuya Scramble Square, al 47° piano, 229 metri sopra la città. Gli orari di apertura vanno dalle 10:00 alle 22:30 e l'ultimo ingresso è consentito alle 21:20.

CLAIM (Hypothesis): L'osservatorio Shibuya Sky si trova al 10° piano.

ETICHETTA VERA: contradiction | PREDETTO DAL MODELLO: entailment

Il modello è vulnerabile alle affermazioni false formulate in modo assertivo e con **alta sovrapposizione lessicale** con il contesto, quando la falsità dipende non dalle parole ma dalla struttura logica. È peraltro un limite noto e condiviso da gran parte dei modelli NLI, non specifico del nostro. La direzione dell'errore, inoltre, è **asimmetrica**: il modello non ha mai bollato come falsa un'affermazione vera, né confuso i neutrali; l'unica perdita è nel verso contraddizione → entailment, cioè proprio quello editorialmente più rischioso.

7. Valutazione e Osservabilità del Sistema

Analisi qualitativa e casi d'uso

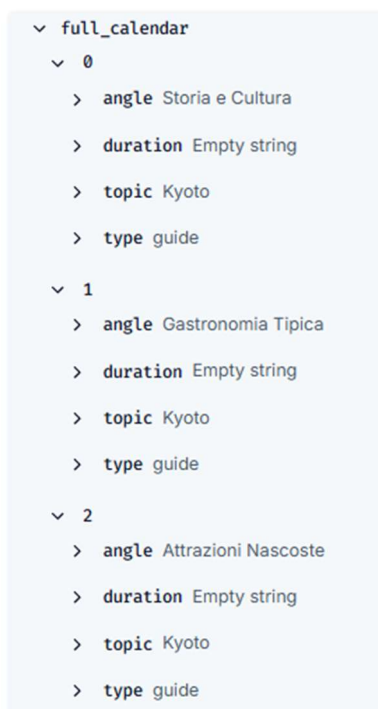
La valutazione qualitativa che segue ha invece un obiettivo complementare: osservare il comportamento dell'intera pipeline agentica dall'inizio alla fine su richieste reali, verificando che l'orchestrazione dei nodi, il grounding K-RAG e i meccanismi anti-allucinazione funzionino in modo coerente lungo tutto il flusso. A tal fine sono stati selezionati due casi rappresentativi e volutamente contrastanti: un caso di successo, in cui la catena si comporta come da progetto, e un caso di fallimento, in cui la rottura di un singolo anello (il collegamento tra claim e dato grezzo) fa degradare la rete di sicurezza fattuale.

Caso d'uso 1 [Es. Guida]

Input dell'utente:

“Scrivimi degli articoli su Kyoto”

Piano editoriale generato:



Ciclo ReACT



researcher

> **System** Sei il Lead Researcher per un'autorevole Guida Turistica...

> **Human** Inizia la tua ricerca autonoma. Usa i tool necessari per...

> **AI** rag_retrieval_tool

> **Tool** rag_retrieval_tool

> **AI** advanced_web_research

> **Tool** advanced_web_research

> **AI** kg_query_tool

> **Tool** kg_query_tool

> **AI** advanced_web_research

> **Tool** advanced_web_research

> **AI** Dopo aver analizzato i risultati del tool, posso conferma...

reasoning_trace

> 0

> 1

action

rag_retrieval_tool({'query': 'Storia, architettura e leggende del tempio Kiyomizu-dera a Kyoto'})

> **agent** researcher_agent

observation

[K-RAG] Query guidata dal Knowledge Graph.

Modalita': thematic_expand

Entita' seed dal KG: Kiyomizu-dera, Kyoto

Termini di espansione: Temple, Kansai, Kyoto, Fushimi Inari Taisha, Foresta di Bambù di Arashiyama, Tempio Tenryuji,

Fiume di Arashiyama

Sotto-query eseguite: ['Storia, architettura e leggende del tempio Kiyomizu-dera a Kyoto', 'Kiyomizu-

thought

Pensando ad alta voce, la mia prima mossa sarà quella di cercare informazioni sulla storia e la cultura di Kyoto, concentrandomi sulle principali attrazioni storiche e culturali della città. Per fare questo, userò il tool rag_retrieval_tool per cercare informazioni specifiche sul tempio Kiyomizu-dera, che è uno dei luoghi più famosi e storici di Kyoto.



researcher

System

Ho già interrogato il Knowledge Graph principale per te. Leggi attentamente cosa sappiamo già:

```
{  
  
  "status": "success",  
  "message": "Collegamenti trovati nel Knowledge Graph.",  
  "krag_context": {  
    "Kyoto": {  
      "entity_type": "City",  
      "name": "Kyoto",  
      "prefecture": "Kyoto",  
      "region": "Kansai",  
      "spots": [  
        "Foresta di Bambù di Arashiyama",  
        "Tempio Tenryūji",  
        "Fiume di Arashiyama",  
        "Kinkaku-ji",  
        "Fushimi Inari Taisha",  
        "Kiyomizu-dera",  
        "Arashiyama Bamboo Forest",  
        "Gion"  
      ],  
      "festivals": [  
        "Gion Matsuri",  
        "Aoi Matsuri"  
      ]  
    },  
    "entities_not_found": null,  
    "errors": null  
  }  
}
```

RECAP

Kyoto: La Città della Storia e della Cultura

INTRODUZIONE E FOCUS EDITORIALE

Kyoto, l'antica capitale del Giappone, è una città ricca di storia e cultura. Con i suoi templi, giardini e festival, Kyoto è un luogo unico e affascinante che attira visitatori da tutto il mondo. In questo dossier, esploreremo la storia e la cultura di Kyoto, scoprendo i suoi segreti e le sue meraviglie.

STORIA E CULTURA

La storia di Kyoto risale al 794, quando la città divenne la capitale del Giappone. Durante il periodo Heian, Kyoto fu il centro della cultura e della politica giapponese. La città fu anche il luogo di nascita del buddhismo giapponese e della cultura samurai. Oggi, Kyoto è una città moderna e vibrante, ma la sua storia e la sua cultura sono ancora molto presenti. I templi e i giardini di Kyoto, come il Kiyomizu-dera e il Fushimi Inari Taisha, sono solo alcuni esempi della ricchezza culturale della città. [Fonti: <https://www.zipangu.it/dove-andare-e-cosa-vedere-a-kyoto/higashiyama/kiyomizu-dera.html>, <https://it.wikipedia.org/wiki/Kiyomizu-dera>]

ATTRAZIONI E COSA VEDERE

- **Kiyomizu-dera:** Il Kiyomizu-dera è un tempio buddhista situato nel cuore di Kyoto. Il tempio è famoso per la sua architettura unica e per la sua storia, che risale al 778. Il Kiyomizu-dera è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo. [Fonte: <https://www.zipangu.it/dove-andare-e-cosa-vedere-a-kyoto/higashiyama/kiyomizu-dera.html>]
- **Fushimi Inari Taisha:** Il Fushimi Inari Taisha è un santuario shintoista situato a Kyoto. Il santuario è famoso per le sue migliaia di porte vermiglie che formano un tunnel lungo la montagna. Il Fushimi Inari Taisha è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo. [Fonte: <https://www.zipangu.it/dove-andare-e-cosa-vedere-a-kyoto/fushimi/fushimi-inari-taisha.html>]
- **Kinkaku-ji:** Il Kinkaku-ji è un tempio buddhista situato a Kyoto. Il tempio è famoso per la sua architettura unica e per la sua storia, che risale al 1397. Il Kinkaku-ji è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo. [Fonte: <https://www.marcotogni.it/tempio-oro-kyoto>]

LOGISTICA E SPOSTAMENTI

Kyoto è una città facilmente raggiungibile con i mezzi pubblici. La stazione di Kyoto è collegata alle principali città del Giappone e offre collegamenti con gli aeroporti di Osaka e Tokyo. La città è anche servita da una rete di autobus e tram che collegano i principali luoghi di interesse. Per raggiungere il Kiyomizu-dera, ad esempio, si può prendere l'autobus numero 100 o 206 dalla stazione di Kyoto e scendere alla fermata di Gojo-zaka. [Fonti: <https://www.giaponismo.com/tempio-kiyomizudera>, <https://enjoy-osaka-kyoto-kobe.com/article/a/kiyomizu-dera-temple-access>]

INFO PRATICHE E PREZZI (DA INCLUDERE OBBLIGATORIAMENTE NEL TESTO)

- Il biglietto per visitare il tempio Kiyomizu-dera costa 500 yen. [Fonte: <https://www.viaggieritratti.it/kiyomizudera>]
- Il tempio Kiyomizu-dera apre al pubblico dalle 6:00 alle 18:00. [Fonte: <https://www.viaggieritratti.it/kiyomizudera>]

FONTI CITATE

- <https://www.zipangu.it/dove-andare-e-cosa-vedere-a-kyoto/higashiyama/kiyomizu-dera.html>
- <https://it.wikipedia.org/wiki/Kiyomizu-dera>
- <https://www.giaponismo.com/tempio-kiyomizudera>
- <https://enjoy-osaka-kyoto-kobe.com/article/a/kiyomizu-dera-temple-access>
- <https://www.viaggieritratti.it/kiyomizudera>
- <https://www.marcotogni.it/tempio-oro-kyoto>
- <https://www.giaponismo.com/kinkaku-ji-tempio-oro>

Fact-checking



fact_checker

fact_check_report

Pagella del Fact-Checker Personale

- **Affermazione:** Il Kiyomizu-dera e' un tempio buddhista situato nel cuore di Kyoto
 - **Contesto analizzato:** *Il Kiyomizu-dera e' un tempio buddhista situato nel cuore di Kyoto. Il tempio e' famoso per la sua architettura unica e per la sua storia, che risale al 778.*
 - **Giudizio LoRA:** ENTAILMENT (Vero)
- **Affermazione:** Il Fushimi Inari Taisha e' un santuario shintoista situato a Kyoto
 - **Contesto analizzato:** *Il Fushimi Inari Taisha e' un santuario shintoista situato a Kyoto. Il santuario e' famoso per le sue migliaia di porte vermiglie che formano un tunnel lungo la montagna.*
 - **Giudizio LoRA:** ENTAILMENT (Vero)
- **Affermazione:** Il biglietto per visitare il tempio Kiyomizu-dera costa 500 yen
 - **Contesto analizzato:** *Il biglietto per visitare il tempio Kiyomizu-dera costa 500 yen.*
 - **Giudizio LoRA:** ENTAILMENT (Vero)
- **Affermazione:** Il tempio Kiyomizu-dera apre al pubblico dalle 6:00 alle 18:00
 - **Contesto analizzato:** *Il tempio Kiyomizu-dera apre al pubblico dalle 6:00 alle 18:00.*
 - **Giudizio LoRA:** ENTAILMENT (Vero)

Articolo generato



▼ drafter

▼ current_draft

Kyoto: La Città della Storia e della Cultura

Mentre cammino per le strade di Kyoto, la città antica e moderna si rivela ai miei occhi come un tesoro di storia e cultura. La luce del sole filtra attraverso le foglie degli alberi secolari, illuminando i templi e i giardini che sembrano sospesi nel tempo. Il profumo di incenso e di fiori di ciliegio mi accompagna in questo viaggio attraverso la città più antica del Giappone.

La Storia di Kyoto

La storia di Kyoto risale al 794, quando la città divenne la capitale del Giappone. Durante il periodo Heian, Kyoto fu il centro della cultura e della politica giapponese. La città fu anche il luogo di nascita del buddhismo giapponese e della cultura samurai. Oggi, Kyoto è una città moderna e vibrante, ma la sua storia e la sua cultura sono ancora molto presenti. I templi e i giardini di Kyoto, come il Kiyomizu-dera e il Fushimi Inari Taisha, sono solo alcuni esempi della ricchezza culturale della città.

 Il tempio Kiyomizu-dera al tramonto, con il Monte Higashiyama sullo sfondo

I Templi di Kyoto

Il Kiyomizu-dera è un tempio buddhista situato nel cuore di Kyoto. Il tempio è famoso per la sua architettura unica e per la sua storia, che risale al 778. Il Kiyomizu-dera è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo. Il biglietto per visitare il tempio Kiyomizu-dera costa 500 yen e il tempio apre al pubblico dalle 6:00 alle 18:00.

Il Fushimi Inari Taisha è un santuario shintoista situato a Kyoto. Il santuario è famoso per le sue migliaia di porte vermiglie che formano un tunnel lungo la montagna. Il Fushimi Inari Taisha è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo.



▼ drafter

▼ current_draft



Il Kinkaku-ji è un tempio buddhista situato a Kyoto. Il tempio è famoso per la sua architettura unica e per la sua storia, che risale al 1397. Il Kinkaku-ji è anche uno dei luoghi più sacri del Giappone e attira visitatori da tutto il mondo. Il biglietto per visitare il tempio Kinkaku-ji costa 500 yen.

Come Muoversi a Kyoto

Kyoto è una città facilmente raggiungibile con i mezzi pubblici. La stazione di Kyoto è collegata alle principali città del Giappone e offre collegamenti con gli aeroporti di Osaka e Tokyo. La città è anche servita da una rete di autobus e tram che collegano i principali luoghi di interesse. Per raggiungere il Kiyomizu-dera, ad esempio, si può prendere l'autobus numero 100 o 206 dalla stazione di Kyoto e scendere alla fermata di Gojo-zaka.





Conclusione

Kyoto è una città che offre una ricchezza di storia e cultura unica al mondo. I suoi templi, giardini e festival sono solo alcuni esempi della sua bellezza e della sua importanza. Se sei un amante della storia, della cultura e della natura, Kyoto è il luogo ideale per te. Non perdere l'opportunità di visitare questa città incredibile e di scoprire i suoi segreti e le sue meraviglie.

Valutazione qualitativa

Il Planner classifica correttamente la richiesta come generica e priva di piani in sospeso, entrando quindi in modalità `new_plan`. La scelta di tre angoli differenti sulla medesima meta è la prova concreta che il vincolo contenutistico imposto all'agente non resta astratto, ma si traduce in una diversificazione reale del calendario, condizione necessaria per prevenire la ridondanza tematica nel tempo.

Il sottografo *Researcher* parte dalla fase di *warmup*, che interroga deterministicamente il Knowledge Graph e recupera un contesto già ricco.

Il ciclo *ReAct* evidenzia il funzionamento nominale dell'agente. Prima di ogni azione l'agente esplicita il proprio ragionamento e alterna in modo autonomo i tool. La `reasoning_trace` mostra che il `rag_retrieval_tool` opera in modalità `thematic_expand`: a partire dal seed *Kiyomizu-dera, Kyoto*, il K-RAG estrae dal grafo termini topologici correlati e li fonde nelle sotto-query, arricchendo la ricerca vettoriale.. È esattamente il comportamento K-RAG previsto in fase di progettazione.

Il nodo *Recap* produce un dossier strutturato e ben partizionato (introduzione, storia e cultura, attrazioni, logistica, info pratiche, fonti citate), in cui ogni sezione e ogni attrazione riportano un URL sorgente reale. Il fatto che tutte le fonti elencate corrispondano a domini editoriali non commerciali (zipangu.it, wikipedia.org, giapponismo.com, marcotogni.it, viaggieritratti.it) conferma che il filtraggio anti-OTA e il Citation Guard hanno operato correttamente, senza citazioni neutralizzate né fonti allucinate.

Il momento decisivo è il fact-checking. Vengono estratti quattro claim e a ciascuno viene associato il paragrafo di contesto esatto dal materiale grezzo. Il modello LoRA classifica tutti e quattro come **ENTAILMENT**, perché per ognuno esiste un contesto grezzo che li supporta. Il Drafter, vincolato a fidarsi solo degli entailment, integra questi dati con sicurezza e produce un articolo che rispetta tutti i controlli del Quality Control Loop: supera le 500 parole,

presenta la struttura H1/H2, include la sezione logistica obbligatoria ("Come Muoversi a Kyoto") ed impagina dinamicamente le immagini recuperate via CLIP.

Il caso dimostra il funzionamento a regime dell'intera architettura: intent detection corretta, diversificazione editoriale reale, espansione K-RAG efficace, isolamento delle fonti e fact-checking pienamente ancorato ai dati grezzi. La condizione che rende il caso un successo è la **continuità del grounding**: ogni affermazione fattuale arrivata nel dossier era rintracciabile nei dati effettivamente recuperati, e questo ha permesso al fact-checker di validarla e al Drafter di usarla. La rete di sicurezza fattuale non è mai stata messa alla prova al limite proprio perché l'informazione è rimasta agganciata alla sua fonte lungo tutta la catena.

Caso d'uso 2 [Es. Itinerario failure case]

Input dell'utente:

“Scrivimi articolo itinerario tra osaka e nara”

Plane:

```
▼ full_calendar
  ▼ 0
    > angle Storia e Cultura
    > duration Empty string
    > topic Osaka, Nara
    > type itinerary
```

Research:

```
📋 ▼ researcher
  > System Sei il Lead Researcher per un'autorevole Guida Turistica...
  > Human Inizia la tua ricerca autonoma. Usa i tool necessari per...
  > AI advanced_web_research
  > Tool advanced_web_research
  > AI advanced_web_research
  > Tool advanced_web_research
  > AI travel_route_tool
  > Tool travel_route_tool
  > AI rag_retrieval_tool
  > Tool rag_retrieval_tool
  > AI advanced_web_research
  > Tool advanced_web_research
```



researcher

System

Stai pianificando un itinerario multi-tappa per 2 destinazioni: Osaka, Nara. Devi chiamare il tool di ricerca

DATI DI PARTENZA (KNOWLEDGE GRAPH):

Ho già interrogato il Knowledge Graph principale per te. Leggi attentamente cosa sappiamo già:

```
{
  "status": "success",
  "message": "Collegamenti trovati nel Knowledge Graph.",
  "krag_context": {
    "Osaka": {
      "entity_type": "City",
      "name": "Osaka",
      "prefecture": "Osaka",
      "region": "Kansai",
      "spots": [
        "Castello di Osaka",
        "Dotonbori"
      ],
      "festivals": [
        "Tenjin Matsuri"
      ]
    },
    "Nara": {
      "entity_type": "City",
      "name": "Nara",
      "prefecture": "Nara",
      "region": "Kansai",
      "spots": [
        "Todai-ji",
        "Parco di Nara"
      ],
      "festivals": [
        "Nara Tokae"
      ]
    }
  }
}
```



AI

AI

Dopo aver valutato i risultati del tool, ho raccolto informazioni utili sull'itinerario tra Osaka e Nara, ma mancano ancora dettagli specifici sulla storia e la cultura di ogni destinazione. In particolare, vorrei approfondire la storia del Castello di Osaka e del Tempio Todai-ji a Nara.

Pertanto, decido di chiamare il tool "rag_retrieval_tool" per cercare informazioni più approfondite sulla storia e la cultura di Osaka e Nara.

rag_retrieval_tool ID

query
Storia e cultura di Osaka e Nara

rag_retrieval_tool

0

[K-RAG] Query guidata dal Knowledge Graph.

Modalita': thematic_expand

Entita' seed dal KG: Osaka, Nara

Termini di espansione: Kansai, Castello di Osaka, Dotonbori, Tenjin Matsuri, Todai-ji, Parco di Nara, Nara Tokae

Sotto-query eseguite: ['Storia e cultura di Osaka e Nara', 'Osaka Kansai', 'Castello di Osaka', 'Osaka Dotonbori']

RECAP:



recap

research_summary

Osaka e Nara: Un Viaggio nella Storia e Cultura del Giappone

INTRODUZIONE E FOCUS EDITORIALE

Osaka e Nara sono due città del Giappone che offrono un'esperienza unica e ricca di storia e cultura. Osaka, con il suo castello e il quartiere di Dotonbori, è una città vibrante e piena di energia, mentre Nara, con il suo tempio Todai-ji e il parco dei cervi, è una città tranquilla e immersa nella natura. In questo dossier, esploreremo la storia e la cultura di queste due città, scoprendo i loro segreti e le loro attrazioni più interessanti.

ITINERARIO GIORNO PER GIORNO

Giorno 1: Osaka

Attività: Osaka è una città che incarna perfettamente il connubio tra passato e modernità. Dalle sue antiche origini come capitale imperiale e snodo commerciale nel Porto di Naniwa, fino alla sua rinascita come metropoli industriale e tecnologica, Osaka ha saputo adattarsi e reinventarsi nei secoli. Il castello di Osaka, costruito nel 1583, è uno dei simboli storici più importanti del Giappone e offre una vista panoramica sulla città.

Logistica: Il castello di Osaka si trova a 10 minuti a piedi dalla stazione Osakajokoen sulla JR Loop Line. Il biglietto d'ingresso costa 600 yen. [Fonti: <https://www.marcotogni.it/castello-osaka>, <https://www.jrailpass.com/blog/it/castello-di-osaka>]

Giorno 2: Nara

Attività: Nara è una città giapponese che vanta una storia ricca e complessa. Durante il periodo Heian, la capitale fu spostata a Kyoto, ma Nara rimase un centro culturale e religioso importante, ospitando molti templi e santuari. Il tempio Todai-ji, costruito nel 752 d.C., è considerato uno dei siti storici più importanti del Giappone e ospita la statua del Grande Buddha.

Logistica: Il tempio Todai-ji si trova nel parco di Nara, a 30 minuti a piedi dalla stazione JR Nara. Il biglietto d'ingresso costa 500 yen. [Fonti: <https://www.marcotogni.it/todaiji>, <https://destinazionegiappone.altervista.org/due-passi-per-nara>]

INFO PRATICHE GENERALI

- Il biglietto d'ingresso per il castello di Osaka costa 600 yen. [Fonte: <https://www.marcotogni.it/castello-osaka>]
- Il biglietto d'ingresso per il tempio Todai-ji costa 500 yen. [Fonte: <https://www.marcotogni.it/todaiji>]



FONTI CITATE

- <https://www.marcotogni.it/castello-osaka>
- <https://www.jrailpass.com/blog/it/castello-di-osaka>
- <https://www.marcotogni.it/todaiji>
- <https://destinazionegiappone.altervista.org/due-passi-per-nara>

Fact-Checking (failure):



fact_checker

fact_check_report

Pagella del Fact-Checker Personale

- **Affermazione:** Il castello di Osaka è stato costruito nel 1583
 - **Contesto analizzato:** *Nessun dato grezzo trovato per questa affermazione*
 - **Giudizio LoRA:** NEUTRO (Non ci sono info sufficienti)
- **Affermazione:** Il biglietto d'ingresso per il castello di Osaka costa 600 yen
 - **Contesto analizzato:** *Nessun dato grezzo trovato per questa affermazione*
 - **Giudizio LoRA:** NEUTRO (Non ci sono info sufficienti)
- **Affermazione:** Il tempio Todai-ji è stato costruito nel 752 d.C.
 - **Contesto analizzato:** *Nessun dato grezzo trovato per questa affermazione*
 - **Giudizio LoRA:** NEUTRO (Non ci sono info sufficienti)
- **Affermazione:** Il biglietto d'ingresso per il tempio Todai-ji costa 500 yen
 - **Contesto analizzato:** *Nessun dato grezzo trovato per questa affermazione*
 - **Giudizio LoRA:** NEUTRO (Non ci sono info sufficienti)

Draft:



drafter

current_draft

Un Viaggio nella Storia e Cultura di Osaka e Nara

Mentre cammino per le strade di Osaka, la città mi avvolge con la sua energia vibrante. Il castello di Osaka, costruito nel 1583, si erge maestoso sulla città, un simbolo storico che racconta la storia di una città che ha saputo adattarsi e reinventarsi nei secoli. La sua architettura imponente e le mura robuste sembrano trasportarmi in un'epoca lontana, dove la storia e la cultura si intrecciano in modo inestricabile.

Giorno 1: Esplorando Osaka

Il mio viaggio inizia a Osaka, una città che incarna perfettamente il connubio tra passato e modernità. Dalle sue antiche origini come capitale imperiale e snodo commerciale nel Porto di Naniwa, fino alla sua rinascita come metropoli industriale e tecnologica, Osaka ha saputo adattarsi e reinventarsi nei secoli. Il castello di Osaka, con il suo giardino e il museo, offre una vista panoramica sulla città e mi permette di comprendere meglio la storia e la cultura di questa metropoli.

Il castello di Osaka visto da lontano, con il giardino e il museo

Per raggiungere il castello di Osaka, posso prendere la JR Loop Line e scendere alla stazione Osakajokoen, che si trova a 10 minuti a piedi dal castello. Il biglietto d'ingresso per il castello costa 600 yen, un prezzo ragionevole per l'esperienza che offre. Mentre esploro il castello, posso anche visitare il quartiere di Dotonbori, noto per le sue luci colorate e la sua atmosfera vivace.

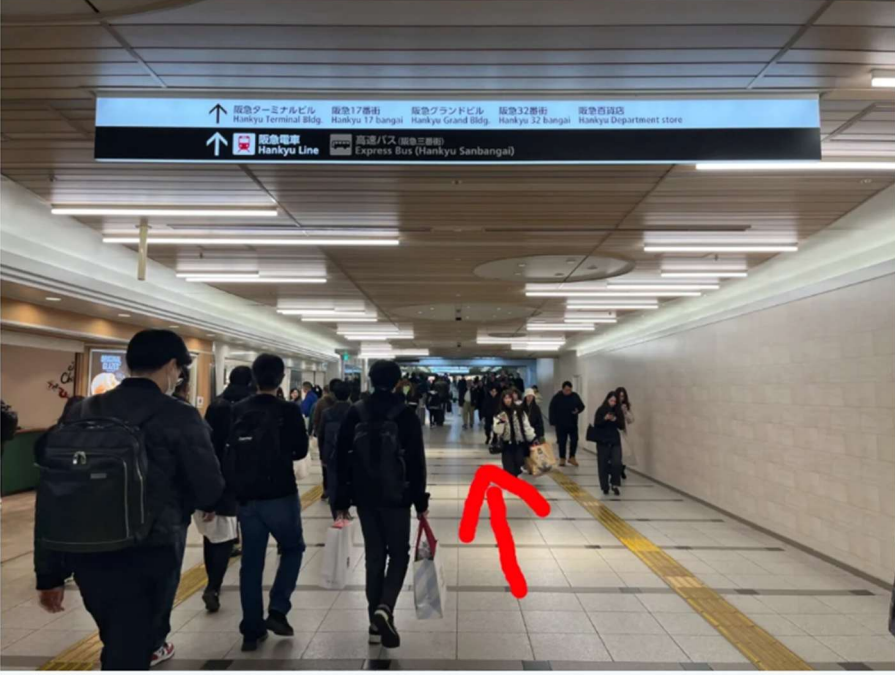
Giorno 2: Scoprendo Nara

Il giorno successivo, mi dirigo verso Nara, una città giapponese che vanta una storia ricca e complessa. Durante il periodo Heian, la capitale fu spostata a Kyoto, ma Nara rimase un centro culturale e religioso importante, ospitando molti templi e santuari. Il tempio Todai-ji, costruito nel 752 d.C., è considerato uno dei siti storici più importanti del Giappone e ospita la statua del Grande Buddha.



▼ drafter
▼ current_draft
Come muoversi

Per muoversi tra Osaka e Nara, posso utilizzare la JR Line, che offre un servizio efficiente e comodo. Il biglietto per il treno può essere acquistato presso la stazione ferroviaria e il prezzo varia a seconda della distanza e del tipo di treno.



In conclusione, il mio viaggio a Osaka e Nara è stato un'esperienza unica e ricca di storia e cultura. Le due città offrono una vista panoramica sulla storia e la cultura del Giappone, e i loro templi, castelli e quartieri storici sono un must-see per chiunque si interessi di storia e cultura. Con il suo connubio di passato e modernità, Osaka e Nara sono due città che meritano di essere visitate e scoperte.

Comportamento osservato

Il Planner riconosce correttamente la natura della richiesta e, applicando la *Regola del Numero di Post*, genera un unico post di tipo itinerary con topic Osaka, Nara. Il warmup del Researcher interroga il Knowledge Graph e recupera con successo entrambe le città (Osaka e Nara, entrambe nel Kansai, con i rispettivi spot e festival). Coerentemente con le direttive condizionali del formato itinerario, l'agente invoca il `travel_route_tool` per la logistica e alterna `advanced_web_research` e `rag_retrieval_tool`. Il Recap produce un `ItineraryDossier` corretto, articolato giorno per giorno con dati pratici e fonti citate.

Fino a questo punto il flusso appare identico a quello del caso di successo. Il fallimento emerge nel nodo di fact-checking. Vengono estratti quattro claim, ma per ciascuno la "Pagella del Fact-Checker" riporta "Nessun dato grezzo trovato per questa affermazione" e un giudizio NEUTRO su tutti e quattro i verdetti.

La causa non è un errore di classificazione del modello LoRA, bensì una rottura di grounding a monte, nella prima fase del fact-checking. L'estrazione dei claim impone che ogni affermazione sia accompagnata da un contesto estratto esclusivamente dal materiale grezzo iniziale: gli output dei tool e i dati storici del Knowledge Graph. In questo caso, le informazioni numeriche puntuali sono comparse nel dossier del Recap senza un corrispondente ancoraggio verbatim nei dati grezzi (dovuto al troncamento delle informazioni principalmente) che il fact-checker interroga. Non trovando un contesto da cui derivarle, l'estrattore consegna

al modello LoRA una premessa vuota; e con una premessa vuota l'unico verdetto possibile corretto dal punto di vista del task NLI è NEUTRO, perché il contesto "non contiene informazioni sufficienti".

È utile notare che il modello LoRA, in questa situazione, si comporta in modo *conservativo e coerente*: in assenza di evidenze non inventa un entailment, ma dichiara di non poter decidere. Il problema non è quindi nel validatore ma nella catena che avrebbe dovuto rifornirlo di evidenze: alcuni fatti sono "trapelati" nel dossier verosimilmente dalla conoscenza parametrica dell'LLM del Recap o da porzioni di sorgente non catturate integralmente.

La conseguenza è duplice. In primo luogo, la rete di sicurezza fattuale degrada silenziosamente: da meccanismo di validazione attiva diventa una serie di "non pervenuto", incapace di confermare o smentire alcunché. In secondo luogo, e in modo più critico, la bozza finale continua a esporre quei dati non validati (il 1583, il 752 d.C., i prezzi), pur essendo classificati NEUTRO e quindi teoricamente da trattare con cautela dal Drafter.

Il fallimento è prezioso perché isola un limite architetturale preciso, coerente con quanto documentato tra i failure case di grounding: il fact-checker è affidabile solo quanto è affidabile il legame tra ciò che finisce nel dossier e ciò che è realmente presente nei dati grezzi. Finché quel legame regge, come nel caso di Kyoto, il sistema valida e protegge; quando si allenta, il sistema non allucina un falso positivo ma smette di garantire, lasciando passare dati non ancorati.

Valutazione delle fonti e grounding

Il rischio principale di qualunque sistema generativo è l'allucinazione: l'invenzione di fatti o di fonti inesistenti. L'architettura adotta diverse contromisure, stratificate lungo l'intera pipeline, per ancorare il testo a informazioni reali e verificabili.

K-RAG (Knowledge Graph + RAG): La ricerca non parte mai da zero. Il nodo Researcher esegue un warmup interrogando il Knowledge Graph sulla meta per recuperare ciò che il blog sa già. Questi dati forniscono un contesto di partenza.

Filtraggio rigoroso nella ricerca web: Il tool `advanced_web_research` accede al web applicando stringenti criteri di qualità. Scarta i risultati con punteggio di rilevanza Tavily inferiore a 0.35. Applica una blacklist di domini commerciali (come `booking.com`, `tripadvisor` o `expedia`) per evitare contenuti promozionali. Infine, filtra i testi con un controllo antispam avanzato e verifica che il contenuto non sia corrotto.

Isolamento degli URL e Citation Guard nel Recap: Il nodo Recap non lascia che l'LLM scriva liberamente i link nel testo. Impone uno schema gerarchico Pydantic in cui ogni dato estratto porta con sé campi dedicati come `source_url` o `source_urls`. Soprattutto, implementa un meccanismo deterministico denominato "Citation Guard". Il sistema estrae preventivamente tramite regex tutti gli URL reali presenti negli output dei tool di ricerca, creando una whitelist.

Se l'LLM prova ad allucinare un URL non presente in questa whitelist, il sistema lo neutralizza, lo aggiunge a una lista di citazioni scartate e imposta l'etichetta di fallback.

Fact-checking come rete di sicurezza: Prima della stesura, i claim fattuali estratti dal dossier vengono confrontati con i rispettivi frammenti di testo originali. Un modello fine-tuned tramite LoRA classifica ogni affermazione in entailment, contraddizione o neutro. Il Drafter è strettamente vincolato a ignorare le informazioni contraddittorie o neutre, basandosi esclusivamente sui fatti valutati come "entailment".

Tracciabilità persistente (KG Updater): L'ancoraggio alle fonti non termina con la generazione dell'articolo. Una volta che il testo viene approvato dall'umano, il nodo `kg_updater` estrae le fonti utilizzate e le storicizza all'interno del Knowledge Graph in Neo4j. Viene creato un nodo specifico per ogni fonte, che viene collegato al nodo dell'articolo tramite la relazione `USED_SOURCE`, garantendo la tracciabilità permanente dei dati.

Analisi dei fallimenti (failure cases)

La robustezza di un sistema si misura anche dalla consapevolezza dei suoi limiti. Di seguito sono documentati tre casi di fallimento, tutti realmente presenti nel comportamento del sistema, insieme alle strategie di mitigazione adottate.

Fallimento 1 — Uscita forzata dal loop dei tool (ReAct). Il nodo `Researcher` è un agente ReAct che decide autonomamente quali tool invocare. Per evitare loop infiniti situazioni in cui l'agente continua a cercare senza mai ritenersi soddisfatto, con costi in tempo e in chiamate API che crescono senza controllo è imposto un hard-limit di 6 chiamate ai tool. Al raggiungimento del limite, il router `route_research` interrompe forzatamente il ciclo e passa alla fase di sintesi. Il fallimento consiste nel fatto che, se in quelle sei chiamate l'informazione cercata non è stata trovata (per esempio perché le fonti web erano di scarsa qualità), l'agente esce comunque, e il dossier prodotto sarà incompleto. Si tratta di un compromesso deliberato: si preferisce un articolo con qualche lacuna a un processo che rischia di non terminare o di esaurire il budget di token.

Fallimento 2 — Limiti della geocodifica (travel_route_tool). Il calcolo degli itinerari multi-tappa si affida al servizio di geocodifica Nominatim (OpenStreetMap) per convertire i nomi di luogo in coordinate. Il tool fallisce quando l'utente o l'LLM che estrae le entità fornisce nomi storpiati, ambigui o non riconosciuti (per esempio l'italianizzazione «Kioto» al posto di «Kyoto», o il nome di un'attrazione minore assente da OpenStreetMap). In questi casi la località viene semplicemente ignorata; se restano meno di due coordinate valide, il tool restituisce un errore strutturato che elenca le località non trovate e suggerisce di usare nomi più precisi in inglese. Il sistema non va in crash e comunica il problema, ma il risultato è la perdita dei dati logistici distanze e tempi di percorrenza per quella tappa, con un impoverimento della sezione «Come muoversi».

Fallimento 3 — Errori di routing. Il terzo caso riguarda le decisioni prese dagli LLM di supporto. L'LLM che classifica il feedback umano può interpretarlo male; in particolare, in

caso di errore delle API (es. un limite temporaneo di Groq) il sistema applica un fallback prudenziale sulla decisione rewrite, che però potrebbe non corrispondere all'intento reale dell'utente (che magari voleva cambiare meta). A questi si aggiunge, come ulteriore fallimento di grounding, l'errore del fact-checker (le contraddizioni «logiche» su Gion e sulle prese elettriche, classificate erroneamente come vere.

LangSmith

La piattaforma di osservabilità LangSmith ha permesso di misurare con precisione i tempi di esecuzione e il consumo di risorse di ciascun nodo del sistema. La tabella riporta i dati relativi a un'esecuzione completa della pipeline per la generazione di un articolo.

Nodo	Latenza (s)	Token consumati
Planner	1.95	440
Researcher	82.24	27.7K
Recap	43.25	9.1K
Fact Checker	26.92	2.2K
Drafter	11.48	4.3K
Human Review	0.63	
KG Updater	1.48	3.0K
TOTALE	~167.95	~47.7K

L'analisi rivela che oltre l'**80% del tempo di esecuzione** è assorbito da tre nodi critici: researcher (82.24s), recap (43.25s) e fact_checker (26.92s). Questo dato empirico conferma che il sottografo di ricerca ReAct rappresenta il principale collo di bottiglia del sistema. La latenza elevata del nodo researcher è attribuibile a molteplici fattori:

1. chiamate sequenziali a tool esterni (Tavily, web scraping, KG query, RAG),
2. ritardi di rete durante lo scraping di pagine web,
3. il limite di 6 chiamate per ciclo di ricerca imposto dalla logica di route_research per prevenire loop infiniti

4. possibili timeout e retry su URL non responsivi.

Il nodo recap, responsabile della distillazione dei dati grezzi in uno schema Pydantic strutturato, richiede 43.25 secondi a causa della lunghezza del contesto da processare e della complessità dell'estrazione, che include anche un post-processing di validazione delle citazioni e deduplica delle fonti. Il nodo fact_checker impiega 26.92 secondi per due operazioni distinte: una prima chiamata LLM per estrarre i claim strutturati (ExtractedClaims), e successive inferenze sul modello LoRA fine-tuned (mDeBERTa) per la verifica di ciascun claim.

I nodi rimanenti presentano tempi di esecuzione contenuti: il planner (1.95s) e il kg_updater (1.48s) sono operazioni leggere di lettura/scrittura su database, mentre il drafter (11.48s) include sia la generazione creativa tramite LLM che le query al modello CLIP per l'impaginazione delle immagini. Il nodo human_review (0.63s) conferma che il meccanismo di interrupt di LangGraph introduce un overhead trascurabile, rendendo l'interazione umana nel loop un meccanismo efficiente per il controllo qualità.

Ottimizzazioni possibili

I dati raccolti suggeriscono tre direzioni prioritarie per l'ottimizzazione:

- ridurre il numero di chiamate ai tool nel ciclo ReAct da 6 a 4, sfruttando il K-RAG per migliorare la pertinenza dei risultati a parità di query;
- adottare un modello più leggero per l'estrazione strutturata nel nodo recap, o implementare una pipeline di distillazione incrementale;
- eseguire il modello LoRA su GPU per abbattere i tempi del fact_checker.